

# ACCELA 编译器技术报告

徐泽逸

黄浩铭

汪子昊

2026 年 7 月

## 目录

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>概述</b>                              | <b>3</b> |
| 1.1      | 总体性能表现 . . . . .                       | 3        |
| 1.2      | 前端与 IR 边界 . . . . .                    | 3        |
| <b>2</b> | <b>LLVM IR 层优化</b>                     | <b>4</b> |
| 2.1      | 公共分析基础设施 . . . . .                     | 5        |
| 2.2      | SROA: 局部/全局聚合标量化 . . . . .             | 5        |
| 2.3      | Mem2Reg: SSA 构建 . . . . .              | 5        |
| 2.4      | EarlyCSE . . . . .                     | 6        |
| 2.5      | SCCP . . . . .                         | 6        |
| 2.6      | InstSimplify 与 InstCombine . . . . .   | 7        |
| 2.7      | SimplifyCFG . . . . .                  | 7        |
| 2.8      | ADCE 与 GlobalDCE . . . . .             | 8        |
| 2.9      | Dead Store Elimination . . . . .       | 9        |
| 2.10     | GVN . . . . .                          | 9        |
| 2.11     | GlobalOpt . . . . .                    | 9        |
| 2.12     | IPSCCP . . . . .                       | 9        |
| 2.13     | 函数内联 . . . . .                         | 10       |
| 2.14     | 尾递归消除 . . . . .                        | 10       |
| 2.15     | 循环规范化与依赖分析 . . . . .                   | 11       |
| 2.16     | Loop Interchange . . . . .             | 11       |
| 2.17     | Loop Rotate . . . . .                  | 11       |
| 2.18     | LICM 与循环内存提升 . . . . .                 | 12       |
| 2.19     | Loop Unroll 与 Unroll-and-Jam . . . . . | 12       |
| 2.20     | IndVarSimplify 与 LFTR . . . . .        | 13       |
| 2.21     | Loop Strength Reduction . . . . .      | 13       |

|          |                                          |           |
|----------|------------------------------------------|-----------|
| 2.22     | Loop Load Elimination . . . . .          | 14        |
| 2.23     | 常量算术 Strength Reduction . . . . .        | 14        |
| 2.24     | 带秩递推表格化 (RRT) . . . . .                  | 15        |
| <b>3</b> | <b>MIR 层优化与寄存器分配</b>                     | <b>16</b> |
| 3.1      | Machine Copy Propagation . . . . .       | 17        |
| 3.2      | PHI Elimination . . . . .                | 17        |
| 3.3      | Memory Address Folding . . . . .         | 18        |
| 3.4      | Machine CSE . . . . .                    | 18        |
| 3.5      | Global Merge . . . . .                   | 18        |
| 3.6      | Machine LICM . . . . .                   | 18        |
| 3.7      | Loop Condition Duplication . . . . .     | 19        |
| 3.8      | Machine Constant CSE . . . . .           | 19        |
| 3.9      | Global Address Materialization . . . . . | 19        |
| 3.10     | Machine Block Placement . . . . .        | 20        |
| 3.11     | 迭代式图着色寄存器分配 . . . . .                    | 20        |
| <b>4</b> | <b>RISC-V Codegen</b>                    | <b>23</b> |
| 4.1      | 指令选择与局部规则 . . . . .                      | 23        |
| 4.2      | 清零 helper 的取舍 . . . . .                  | 23        |
| 4.3      | 浮点/整型混合与舍入 . . . . .                     | 24        |
| <b>5</b> | <b>性能与正确性分析工具</b>                        | <b>24</b> |
| 5.1      | 分析工具与证据边界 . . . . .                      | 24        |
| 5.2      | AST 解释器与差分正确性基线 . . . . .                | 24        |
| 5.3      | QEMU TCG 动态分析工具 . . . . .                | 24        |
| 5.4      | llvm-mca 与 BOOM v3 成本估算 . . . . .        | 25        |
| 5.5      | ACCELA 与 LLVM -O3 的 60 点横评 . . . . .     | 25        |
| <b>6</b> | <b>AI 使用与优化合法性分析</b>                     | <b>29</b> |
| 6.1      | AI 使用说明 . . . . .                        | 29        |
| 6.2      | Pass 的合法性分类 . . . . .                    | 31        |
| 6.3      | 部分测试点优化结果复核 . . . . .                    | 35        |
| <b>7</b> | <b>结论</b>                                | <b>36</b> |

## 1 概述

ACCELA 是一个由 Java 开发的 Sysy 编译器, 它接受 SysY2022 源程序, 输出 RV64GC GNU 汇编。架构上, ACCELA 建立了显式的 SSA LLVM IR/Machine IR 双层级优化流水线: 中端负责消除可证明冗余的计算和内存访问, 后端负责指令选择、布局优化、全局地址复用、基于图着色分配的寄存器分配与汇编代码生成。图 1 按当前 AAAC 的真实执行顺序列出主要步骤, 值得注意的是同名的 Pass 在流水线中可能重复运行, 用于清理前一轮变换暴露出的机会。

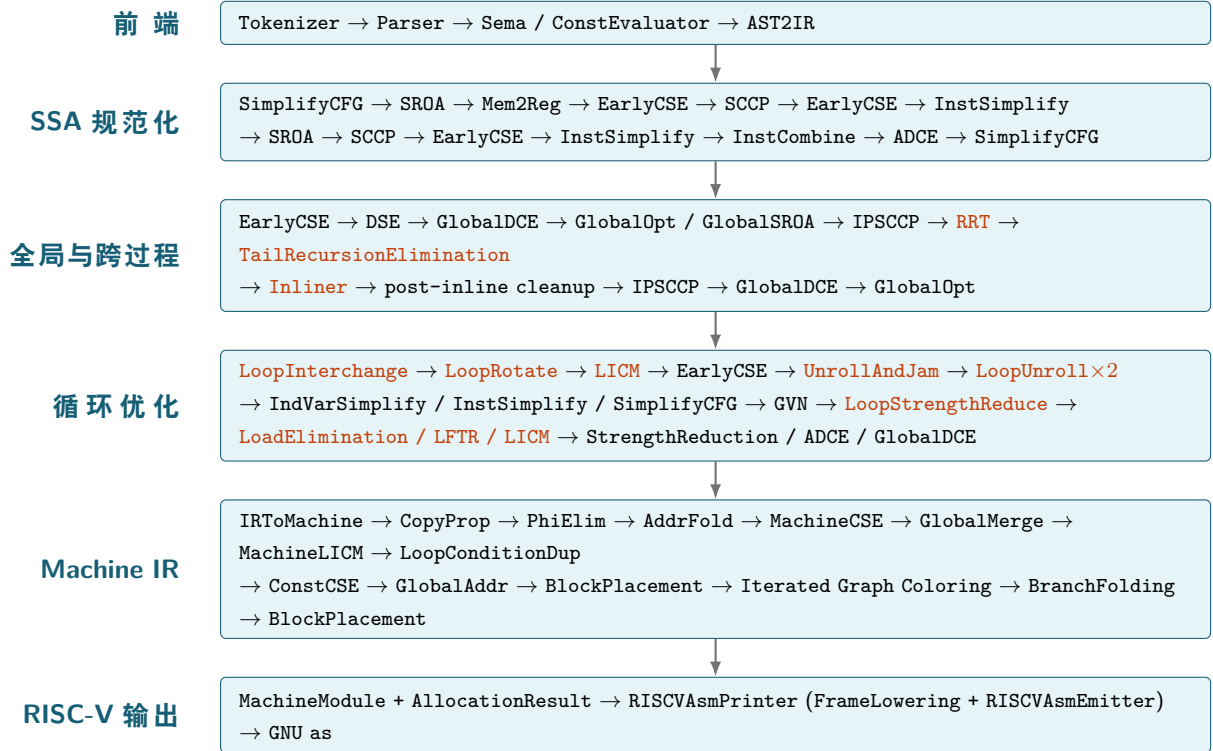


图 1: ACCELA 当前优化流水线, 橙色文字表示按合法性或收益条件选择性触发的步骤

### 1.1 总体性能表现

在初赛版本 60 个 perf 点的 QEMU 动态指令比较中, ACCELA 相对 LLVM -O3 胜 31 点、负 29 点; 以  $R = N_{LLVM}/N_{ACCELA}$  定义指令数比, 60 点几何平均  $R = 1.653$ , 总指令数为 8.666B 对 15.119B。完整 instruction/load/store 数据、分布图统一列于第 5 节。

### 1.2 前端与 IR 边界

**Lexer.** ACCELA 的词法分析器采用手写单遍扫描, 将源文件转换为扁平 Token 流。空白、单行注释和块注释在这一阶段被跳过; 标识符扫描完成后通过关键字表区分 `int`、`float`、`const` 和控制流关键字, 多字符运算符 `<=`、`==`、`&&`、`||` 等也在此阶段合并。数值 Token 暂时保留源码拼写, 因此十进制、八进制和十六进制整数, 以及带小数、指数或后缀的十进制与十六进制浮点数, 不会在 Lexer 中分散处理。Lexer 只负责确定一个数值 Token 的边界, 未知字符则立即作为词法错误报告。

**Parser.** Parser 对声明和语句使用手写的递归下降实现，对表达式使用基于优先级表的 Pratt 解析。乘除模、加减、关系、相等和短路逻辑运算按绑定强度构造左结合 AST；赋值被单独处理为最低优先级的右结合运算。函数、形式参数、词法块、变量声明、if/else、while、break/continue 和 return 最终都表示为统一的带标签 Node，避免为每一种语法形式建立独立的 Java 类。

Parser 有意保留尚未确定的源语言结构：数组维度仍保存为表达式，数组形参省略的首维通过标志记录，花括号初始化按源码嵌套关系构造原始 INIT\_LIST，连续下标则先形成嵌套的 SUB 节点。这些结构交由 Sema 根据类型和常量语义进一步规范化。数值 Token 是一个例外：Parser 在建立字面量节点时调用 LiteralValue.parse，去除允许的类型后缀，识别十进制、八进制、十六进制整数以及十进制、十六进制浮点数，并统一转换为 32 位 int 或 float。此后 Sema、常量求值器和 AST2IR 均直接读取带类型的 LiteralValue，不再重复解析字符串。

**Sema 与常量语义.** Sema 不仅进行传统类型检查，还承担名字绑定和 AST 规范化。它使用具有父指针的嵌套符号表表示词法作用域，并预先注册 SysY 输入输出及计时运行库函数；每个 REF 和直接调用节点都绑定到对应声明，因而后续阶段不必再次按名字解析。

Sema 同时计算字面量、引用、调用、一元运算、二元运算和数组下标的源语言类型：逻辑运算、关系运算和逻辑非的结果统一为 int；混合整浮点算术、赋值以及具有已知形参类型的函数调用会插入显式 CAST 节点。这样，源码中的隐式转换在 AST 上成为可见操作，解释器、常量求值和 IR 生成可以共享同一套转换语义。

数组维度在 Sema 中被求值为整数，并与元素类型组合成携带完整维度信息的 Ty.array。解析阶段产生的 SUB(SUB(a,i),j) 会被展平为 SUB(a,i,j)，其结果类型通过逐层去除数组维度得到；若被访问对象是常量数组且所有下标均为编译期常量，Sema 还可沿初始化树直接取得对应叶节点，这些规范化使 AST2IR 无需重新处理源语言的声明形状和多维下标语法。

数组初始化也是 Sema 的主要规范化工作。非空聚合首先按数组总元素数建立线性缓冲区，并以零预填；随后依据 SysY/C 的子聚合对齐规则填入原始花括号内容，对每个标量元素完成语义分析，最后重建维度完整的嵌套 INIT\_LIST。因此部分初始化、嵌套花括号和省略元素到达 AST2IR 时具有确定布局。标量 const 初始化则直接折叠成目标类型的字面量。空初始化列表 {} 特意不展开，仍以没有子节点的 INIT\_LIST 紧凑表示，避免超大零数组在前端产生数千万个 AST 节点。

SysY 的 starttime/stoptime 在 AST2IR 中被显式改写为 \_sysy\_starttime/\_sysy\_stoptime，并补入运行库所需的行号参数。后续别名/内存效应分析将这两个调用标记为可观察副作用，因而不得被删除、跨越或重排，我们在第 6 节另行给出完整的合法性审计。

## 2 LLVM IR 层优化

ACCELA IR 采用了和 LLVM IR 一致的设计，包括带类型的 SSA Value、显式基本块和 def-use 链，但它是独立实现的 Java IR，并非 LLVM IR 的 Java binding。采用这一设计，一方面可以将 LLVM 作为语义参照，对同一 SysY 程序执行差分测试，并借助相近的指令语义快速定位结果差异；另一方面可以复用 LLVM 工具生态，例如使用 llvm-mca 分析 ACCELA 生成的 RISC-V 热点汇编，并与 QEMU 动态事件和真实硬件耗时交叉验证。完整的性能与正确性分析工具链见第 5 节。

## 2.1 公共分析基础设施

ACCELA 为 IR 优化提供了一组共享的控制流分析。支配树描述一条定义或控制条件必然生效的区域, 并通过 Dominance Frontier 支持 Mem2Reg 的  $\phi$  插入; LoopAnalysis 在此基础上识别自然循环及其 header、latch、preheader 和嵌套关系, 供 LICM、循环变换与 RRT 使用。Induction-VariableAnalysis 进一步识别由初值和固定步长回边构成的归纳变量, PostDominatorTreeAnalysis 则描述从基本块到函数出口的必经关系, 为 ADCE 判断控制依赖和保留必要分支提供依据。

内存分析回答“两个地址是否可能指向同一对象”以及“函数调用可能读写哪些对象”两个问题。PointerProvenance 沿 GEP 追溯指针的根对象, 区分不同的全局变量、局部栈槽和函数参数; GlobalModRefAnalysis 汇总每个函数对全局对象及指针参数的读写行为, 并沿直接调用关系迭代传播到调用者。对于无法证明的外部调用或指针关系, 分析保守地假定可能发生读写; 只有能够证明对象相互独立时, GVN、EarlyCSE、LICM 和 DeadStoreElimination 才能跨越其他访存或调用保留、移动或删除内存操作。

这些分析由 FunctionAnalysisManager 按需计算并按函数缓存。IR Pass 修改 CFG 或 def-use 关系后, 未被显式保留的结果会失效并重新计算, 从而在避免重复分析的同时保证后续优化不会使用过期信息。

## 2.2 SROA: 局部/全局聚合标量化

**算法描述** 局部 SROA 先把只被常量 GEP 访问、地址不逃逸的数组 `alloca` 拆成独立标量槽, 再把可直接提升的参数槽和标量 `alloca` 交给 Mem2Reg。模块级入口对小型全局数组做同样的叶节点拆分。扫描和重写为  $O(U)$ ; 类型树展开的最坏大小受叶节点阈值限制。

例如, 源程序中的 `int a[2] = {}; a[1] = x; return a[1] + 1;` 最初需要通过数组对象和 GEP 访问元素。SROA 证明地址没有逃逸且下标恒为 1 后, 只为实际访问的叶节点建立标量槽; 紧接着的标量提升又可消除该槽。为展示两步之间的关系, 图 2 把实际连续执行的过程拆成三个阶段:

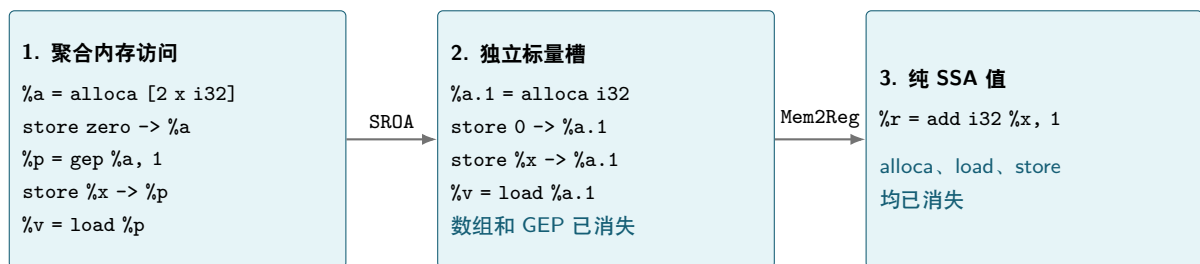


图 2: 局部数组从聚合内存访问到纯 SSA 值的简化过程

如果下标在编译期不确定, 或者数组地址被传给其他函数, SROA 无法建立这种“一个 GEP 对应一个叶节点”的映射, 原聚合对象会保持不变。

## 2.3 Mem2Reg: SSA 构建

**算法描述** 单基本块 `alloca` 走线性转发快路径, 跨块 `alloca` 先在定义块的迭代 Dominance Frontier 插入 PHI, 再沿 Dominator tree 以值栈重命名 load/store, 算法与 Cytron 等人的经典 SSA 构造

一致 [3]。复杂度为  $O(N + E + \Phi)$ ，其中  $\Phi$  为新建 PHI 的输入数。

## 2.4 EarlyCSE

**算法描述** 对每个基本块维护表达式哈希表和可用 load 表；相同 opcode、类型、谓词和操作数复用先前 SSA value。Store 只杀死可能别名的 load，call 按 GlobalModRefAnalysis 的写集合失效，纯 call 也可 CSE。平均复杂度  $O(N)$ ；若指针来源退化，别名过滤最坏为  $O(N^2)$ 。

图 3 展示同一基本块内的典型过程。第二次读取前没有可能写入 %p 的 store 或 call，因此它与第一次 load 处于同一抽象内存版本；随后的 add 也具有相同的 opcode、类型和操作数。

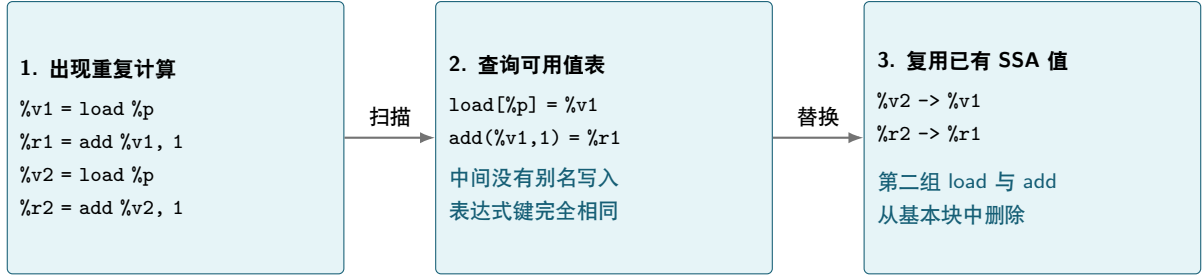


图 3: EarlyCSE 对重复 load 和纯表达式的消除过程

**正确性论证** 纯表达式仅在同块、同操作数且先前定义支配当前位置时复用，Load 只有在中间不存在可能写入别名对象的 store/call 时复用，故读取的抽象内存版本一致。

## 2.5 SCCP

**算法描述** SCCP 在 BOTTOM/constant/TOP 三点格上同时传播 SSA 值和可执行 CFG 边，条件成为常量后只访问可行后继。固定点后，它替换已证明的常量，并把常量条件分支改写为无条件边；后续 SimplifyCFG 再删除由此暴露的不可达块。这是 Wegman-Zadeck 稀疏条件常量传播的受限实现 [5]。格高度固定，故工作量为  $O(N + E + U)$ 。

图 4 中，常量比较使 else 边不可执行，因此汇合点的 PHI 只接收 then 路径上的值 40，后续加法也随之折叠为 42。

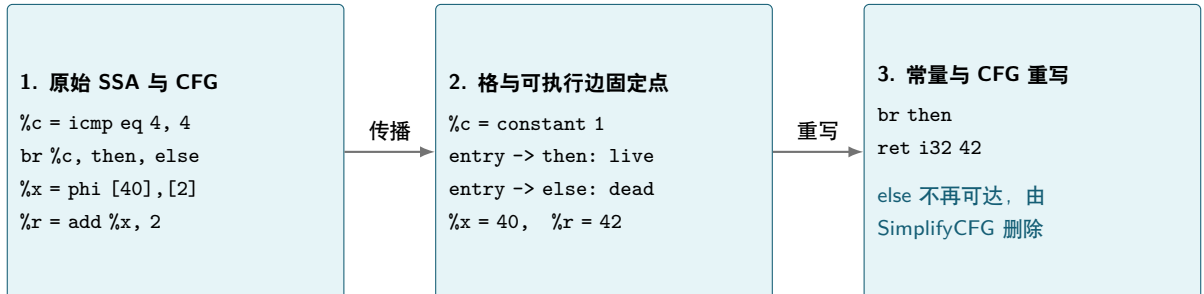


图 4: SCCP 联合传播常量与可执行 CFG 边的过程

**正确性论证** 只有所有可执行到达路径在格上汇合为同一常量时才替换值；不可执行块必须先由已证明的常量分支排除，因此变换不删除任何可达行为。



## 2.6 InstSimplify 与 InstCombine

**算法描述** InstSimplify 用 worklist 反复应用不新增指令的局部规则，包括整数常量折叠、代数恒等式、布尔比较化简、常量分支改写和无副作用死指令删除。InstCombine 处理需要查看小型表达式树或控制流前驱的组合：加减树消项、布尔 PHI 窄化、二次幂余数测试以及已证明整除路径上的除法。表 1 列出当前实现的完整规则族。

| 阶段           | 规则族       | 当前实现中的变换与触发条件                                                                                                                                                                                                                      |
|--------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| InstSimplify | 整数常量折叠    | add/sub/mul/sdiv/srem/shl/and/xor、整数比较及 zext/sext 的操作数均为常量时直接求值；除数必须非零，移位量必须在 $[0, 31]$ 。当前出于实现复杂度考虑不折叠浮点算术与浮点比较。                                                                                                                  |
| InstSimplify | 代数恒等式     | $x+0 \rightarrow x$ 、 $x-0 \rightarrow x$ 、 $x-x \rightarrow 0$ 、 $x \times 1 \rightarrow x$ 、 $x \times 0 \rightarrow 0$ 、 $x/1 \rightarrow x$ 、 $x \ll 0 \rightarrow x$ 、 $x \& x \rightarrow x$ 、 $x \oplus x \rightarrow 0$ 等。 |
| InstSimplify | 布尔、CFG 相关 | zext i1 %c != 0 还原为 %c。常量条件分支改写为无条件分支。                                                                                                                                                                                             |
| InstCombine  | 线性加减树     | 把 i32 add/sub 树收集为“有符号项 + 常量”，合并常量并消去正负各出现一次的项，例如 $(a+b)-(a+c) \rightarrow b-c$ 。                                                                                                                                                  |
| InstCombine  | 布尔 PHI 窄化 | icmp ne (phi i32 [0, zext i1 %c]), 0 变为 phi i1 [false, %c]。PHI 必须只有该比较一个 use，所有 incoming 值必须是 0、1 或 zext i1。                                                                                                                       |
| InstCombine  | 二次幂余数测试   | $(x \text{ srem } 8) == 0$ 变为 $(x \& 7) == 0$ 。                                                                                                                                                                                    |
| InstCombine  | 精确二次幂除法   | 若进入当前块的唯一边已经证明 $x \text{ srem } 8 == 0$ 或 $(x \& 7) == 0$ ，则在该块内把 $x \text{ sdiv } 8$ 改为 $x \text{ ashr } 3$ 。没有整除证明时不进行该变换。                                                                                                       |

表 1: InstSimplify 与 InstCombine 当前实现的规则

InstSimplify 的单条规则为常数时间，整体受 worklist fuel 限制。InstCombine 对一个加减树最多访问 64 个节点，并扫描到局部固定点，通常工作量接近  $O(64N)$ ，若每轮全函数扫描只成功改写一个表达式，保守上界为  $O(64N^2)$ 。

**正确性论证** InstSimplify 只用已有值或同类型常量替换结果；除零和越界移位不会折叠，删除指令前还要求结果无 use 且操作无副作用。InstCombine 的加减树只合并 i32 单位系数项并要求成本下降；布尔 PHI 只接收可证明为 i1 的 incoming value，余数掩码显式处理符号位；除法转移位则要求唯一前驱边已经证明被除数可被相应二次幂整除。因此每项重写都保持原有定宽整数和 signed 运算语义。

## 2.7 SimplifyCFG

**算法描述** 该 Pass 删除不可达块、折叠常量/空转分支、合并单前驱块、修复 PHI edge，并实现短路分支线程化、boolean PHI diamond 折叠和 tail merge。每轮扫描为  $O(N+E)$ ，局部重写后继续到固定点，同时我们在实测中发现该 Pass 在实际 SysY CFG 中运行成本很小。

图 5 给出 boolean PHI diamond 的典型形态。条件块的一条边直接到达汇合块，另一条边经过仅含无条件跳转的空转块，若汇合处的 i32 PHI 恰好在两条边上选择 0 和 1，就可直接以分支

条件的 `zext` 取代 PHI。若 0、1 的次序相反，则先对条件取反再扩展。

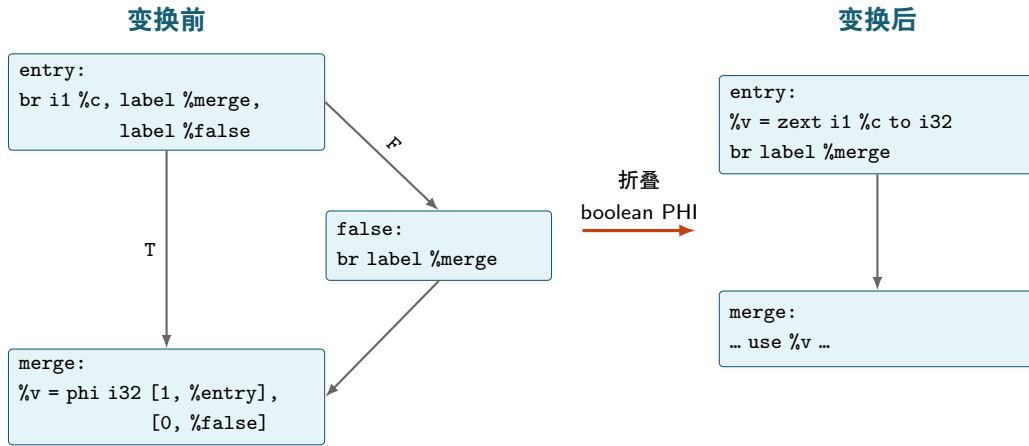


图 5: SimplifyCFG 折叠 boolean PHI diamond。矩形表示基本块，箭头表示 CFG 边。

图 6 展示当前 tail merge 的范围：两个候选块分别只含一条结构等价的纯计算和一条指向相同后继的无条件分支。只要后继 PHI 的 incoming value 也可相应合并，Pass 就保留其中一个块，把另一块的前驱边重定向到它，并以保留值替换被删除值；随后单入口 PHI 折叠会消去多余的 PHI。该实现有意不匹配任意长度的公共指令后继。

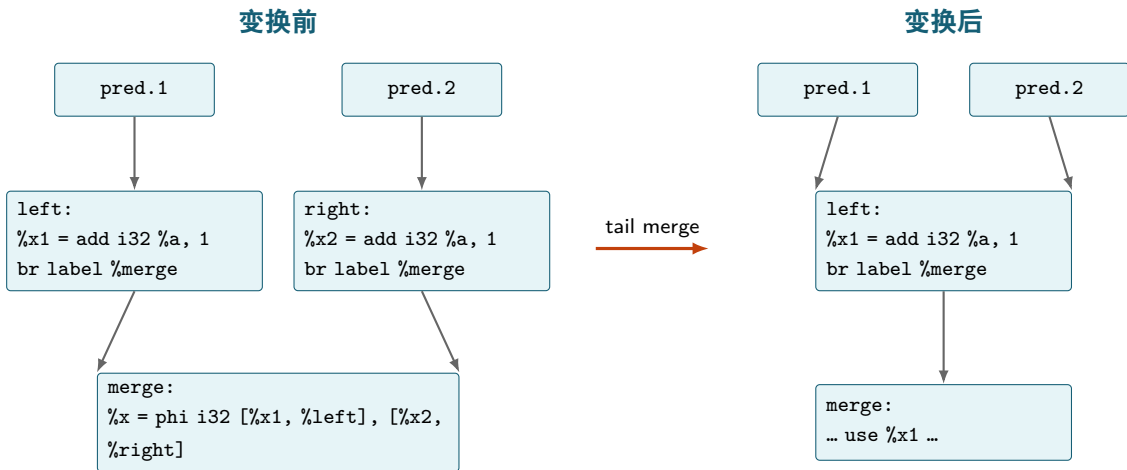


图 6: SimplifyCFG 合并结构等价的尾块，并修复后继 PHI。

## 2.8 ADCE 与 GlobalDCE

**算法描述** 函数级 ADCE 从 `return`、可能写内存的 `store/call` 等 root 反向标记数据依赖，再利用 post-dominance 保留影响活跃控制流的分支，模块级入口从 `main` 遍历 direct-call graph，删除不可达函数和无引用全局对象，复杂度为  $O(N + E + U)$ 。

**正确性论证** 删除的指令既不贡献任何活跃 SSA 值，也不具有可观察内存效果，删除的内部函数不在从 `main` 可达的调用图中，故程序输出不可能观察到它们。



## 2.9 Dead Store Elimination

**算法描述** 模块级 DSE 用对象来源与 MemoryEffects 追踪 store 之后到下一次可能读取之间的写覆盖；被完全覆盖且中途无别名 load/call 的旧 store 可删除。保守扫描的最坏复杂度为  $O(N^2)$ ，在实际测试中我们发现每个对象的局部 store 链通常很短。

## 2.10 GVN

**算法描述** GVN 沿 dominator tree 维护有作用域的 value-number 表，消除跨基本块的支配表达式和内存版本相同的 load，PHI translation 还能把各前驱末尾相同的二元表达式提升到 merge 块一次计算。平均复杂度  $O(N + U)$ ，别名查询退化时最坏  $O(N^2)$ 。

图 7 展示 PHI translation。当前实现要求每个 incoming value 都是在对应前驱中定义且仅由该 PHI 使用的纯二元指令，同时这些指令的 opcode、类型和两个操作数完全相同。满足条件时，两条路径上的重复计算被删除，等价表达式改为在汇合块中计算一次。

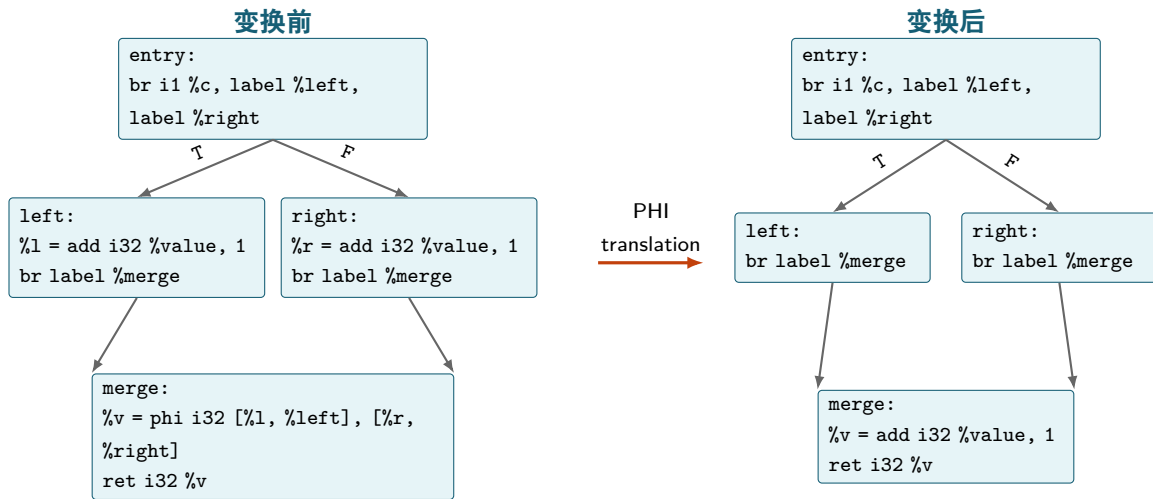


图 7: GVN 通过 PHI translation 将两条前驱路径上的相同表达式移到汇合块中计算一次。

## 2.11 GlobalOpt

**算法描述** GlobalOpt 将只在 main 中直接使用的内部标量全局变为局部 alloca 并立即 Mem2Reg。对由 helper 读取、调用点可追踪的标量全局，则把值提升成显式函数参数，随后重新运行 SSA 清理。内联后再执行一次，覆盖原先跨调用不可见的机会。复杂度近似为模块 def-use 和调用边总数的线性函数。

## 2.12 IPSCCP

**算法描述** solver 为每个函数维护 executable 标志、参数格和返回值格，从 main 出发在调用图与函数内 SCCP 间迭代固定点，完成后用已知参数/返回值重写函数，再跑 SimplifyCFG。格高度有限，最坏为调用边与 IR 边反复激活，实际工作量近似  $O(F(N + E))$ 。

### 2.13 函数内联

**算法描述** 模块调用图上最多进行四轮扫描，合法 call site 克隆 callee CFG、参数替换为实参、return 汇入 continuation PHI，随后清理 CFG。成本估计以 callee 指令数为主，最坏复制量受站点数和 caller 大小共同限制。

图 8 给出一个双返回 callee 的内联例子。形参 %c 被调用点的实参 %flag 替换，原 call 之后的指令被移入 continuation，两个克隆 return 则改为跳向 continuation 并通过 PHI 产生原 call 的结果。

| 内联前                                                                                                                                                                      | 内联后                                                                                                                                                                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>define i32 @choose(i1 %c) { entry:   br i1 %c, label %yes, label %no yes:   ret i32 1 no:   ret i32 2 }  %r = call i32 @choose(i1 %flag) %sum = add i32 %r, 3</pre> | <pre>br label %choose.entry.inline choose.entry.inline:   br i1 %flag, label %choose.yes.inline,     label %choose.no.inline choose.yes.inline:   br label %continuation choose.no.inline:   br label %continuation continuation:   %r = phi i32 [1, %choose.yes.inline],     [2, %choose.no.inline]   %sum = add i32 %r, 3</pre> |

图 8: 函数内联对参数、控制流和返回值的改写。

### Policy 量化

| 规则               | 当前值       |
|------------------|-----------|
| 最大轮数             | 4         |
| 单次调用 callee 成本上限 | 240 条 IR  |
| 重复调用 callee 成本上限 | 80 条 IR   |
| 内联后 caller 上限    | 2000 条 IR |

### 2.14 尾递归消除

**算法描述** 直接自递归且递归 call 紧邻 return 时，把入口参数改成 header PHI，并将尾调用实参接到回边，原 call/return 变成跳转。扫描为  $O(N + E)$ ，额外 PHI 数等于参数数。

图 9 对比两个直接自递归片段。这里假定函数入口没有前驱且函数内没有 alloca，左侧递归 call 的唯一 use 是紧邻的 return，因此可以消除，右侧仍需对递归结果执行加法，因此不能改写为回边。

## 可以消除

```
tail:
  %next = sub i32 %n, 1
  %acc2 = add i32 %acc, %n
  %r = call i32 @sum_tail(
    i32 %next, i32 %acc2)
  ret i32 %r
```

%next 和 %acc2 成为 header PHI 的回边，call 和 return 被 br label %header 取代。

## 不能消除

```
recurse:
  %next = sub i32 %n, 1
  %r = call i32 @sum(i32 %next)
  %v = add i32 %r, %n
  ret i32 %v
```

递归 call 与 return 之间还有 add，返回后必须继续执行当前调用中的工作，改成回边会丢失这次加法。

图 9: 尾递归消除可以触发和不能触发的例子。

## 2.15 循环规范化与依赖分析

循环优化共享统一的循环结构、归纳变量和依赖分析结果。LoopAnalysis 识别 header、pre-header、latch、exit 与嵌套关系，InductionVariableAnalysis 提取 canonical IV，并在边界可静态确定时计算精确 trip count。对需要重排迭代次序的 interchange 和 unroll-and-jam，依赖分析将 GEP 地址表示为循环 IV 的仿射式，计算迭代距离和方向向量，只有能够证明变换不会颠倒数据依赖或引入访存冲突时才执行，否则保守拒绝。

## 2.16 Loop Interchange

**算法描述** 在紧密嵌套、规范化的二重循环上交换 header、latch 和 PHI 的角色。DependenceAnalysis 要求交换后的方向向量不出现逆序依赖，profitability 检查交换是否改善最内层地址步长。候选发现和依赖检查最坏为  $O(A^2)$ ， $A$  为循环内访存数。

| 状态  | 循环顺序                                                        | 内层相邻访问                          | i32 地址步长 |
|-----|-------------------------------------------------------------|---------------------------------|----------|
| 交换前 | for i in [0, N)<br>for j in [0, M)<br>B[j][i] = A[j][i] + 1 | $A[j][i] \rightarrow A[j+1][i]$ | 4N 字节    |
| 交换后 | for j in [0, M)<br>for i in [0, N)<br>B[j][i] = A[j][i] + 1 | $A[j][i] \rightarrow A[j][i+1]$ | 4 字节     |

表 2: Loop Interchange 将连续访问移入最内层循环。

## 2.17 Loop Rotate

**算法描述** 把适合的入口判定循环旋转为带入口 guard 的 latch 判定形式。入口 guard 保留第一次条件检查以维持零次迭代语义，header 中可安全复制的条件计算被复制到 latch，循环后续迭代由 latch 直接跳回 body。该结构便于后续 LFTR、load elimination 和 block fallthrough，候选识别与 CFG 重写为  $O(N + E)$ 。

图 10 展示结构变化。旋转前每次迭代都要从 latch 回到 header 再判断，旋转后入口 guard 只

负责首次判断, latch 使用更新后的 IV 重新计算条件并直接选择 body 或 exit。为突出关键 CFG 边, 图中省略了实现生成的仅含无条件跳转的 .lr.ph 中转块。

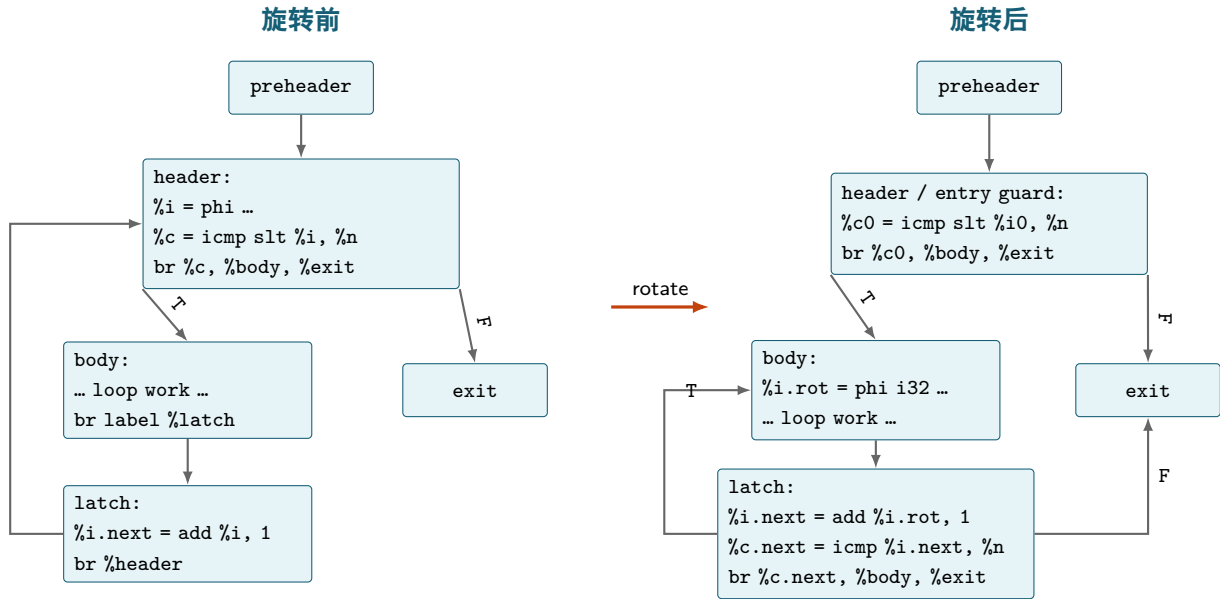


图 10: Loop Rotate 将循环的重复条件判断从 header 移到 latch, 同时保留入口 guard。

## 2.18 LICM 与循环内存提升

**算法描述** 在 preheader 存在时, 反复提升操作数已在循环外可用且安全执行的纯指令, load 还需 PointerProvenance/ModRef 证明循环内无别名写。标量对象可进一步在 preheader load、循环内 SSA PHI、exit store 之间提升。最坏别名比较为  $O(N^2)$ , 常见情形接近线性。

## 2.19 Loop Unroll 与 Unroll-and-Jam

**算法描述** 两者都通过复制循环体减少控制流开销并暴露后续标量优化机会, 但复制的维度和合法性条件不同。Loop Unroll 面向 trip count 精确已知的最内层循环并完全展开, Loop Unroll-and-Jam 则面向二重循环, 复制相邻 outer iteration 后把各 lane 的 inner body 合入同一个 inner loop。

| Pass           | 候选与合法性                                        | 变换方式                                                                                        | 收尾与成本                                                                             |
|----------------|-----------------------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Loop Unroll    | 常量 trip count 为 $T$ 的 innermost loop          | 串接 $T$ 份克隆迭代, 用最后一轮 header value 替换 live-out                                                | 完全展开后运行 SCCP、InstSimplify、ADCE 和 SimplifyCFG, 成本为 $O(TN)$ 。每次调用处理一个候选, 默认管线连续调用两次 |
| Unroll-and-Jam | perfect outer/inner nest, 仿射依赖分析证明相邻 lane 不干扰 | 按因子 $f$ 复制 outer iteration, 把各 lane 的 inner body 合入同一 inner loop, 并为每个 lane 建立独立 scalar PHI | 保留原顺序的 remainder loop 处理不能被 $f$ 整除的 outer iteration, 成本为 $O(A^2 + fN)$            |

表 3: Loop Unroll 与 Unroll-and-Jam 的适用范围和变换方式。

## 2.20 IndVarSimplify 与 LFTR

**算法描述** IndVarSimplify 识别  $i = 0, i < \text{bound}, i += 1$  形式的 canonical IV, 并把入口条件蕴含的  $0 \leq i < \text{bound}$  传播到受循环 body 支配的指令。当前实现只折叠可证明恒真的 signed 比较, 并在仿射范围证明操作数非负且不溢出时把正二次幂 srem 改为位掩码。LFTR 在后续阶段匹配带入口 guard 的旋转循环及与整数 IV 对齐的单位步长 pointer PHI, 在 preheader 计算终点指针, 再把 latch 的  $i.\text{next} < \text{bound}$  改为  $p.\text{next} \neq p.\text{end}$ 。若  $L$  为可识别归纳变量数, 当前直接实现的范围查询最坏为  $O(NL)$ , LFTR 对单个候选沿 PHI 和 use 链扫描为  $O(N + U)$ 。

| 规则    | 变换前                                  | 变换后                                | 关键前提                                                  |
|-------|--------------------------------------|------------------------------------|-------------------------------------------------------|
| 范围比较  | <code>icmp slt (i - 2), bound</code> | <code>true</code>                  | 指令位于由 $i < \text{bound}$ 控制的 body 中                   |
| 二次幂取余 | <code>(5*i + 1) srem 16</code>       | <code>(5*i + 1) &amp; 15</code>    | 例如 $0 \leq i < 64$ 足以证明被除数非负且落在 signed i32 内          |
| LFTR  | <code>icmp slt i.next, bound</code>  | <code>icmp ne p.next, p.end</code> | 已旋转且有入口 guard, pointer PHI 与整数 IV 同步递增, 整数 IV 在改写后可删除 |

表 4: IndVarSimplify 与 LFTR 当前实现的代表性改写。

## 2.21 Loop Strength Reduction

**算法描述** 该 Pass 在无 call 的 canonical loop 中寻找基址不变、恰有一个 index 随 IV 仿射变化且最终到达 load 或 store 的 GEP。它在 preheader 计算首迭代地址, 在 header 建立 pointer PHI, 并在 latch 以固定 element step 生成  $p.\text{next} = \text{gep } p, \text{step}$ , 从而用一次递推取代每轮的 index

扩展、乘法和多层 GEP。结构相同的地址共享同一 recurrence，符号项相同且只差小常量偏移的直接访存可以从已有 recurrence 派生。单位步长 recurrence 在整数 IV 不再有其他有效 use 时还可把退出条件改为终点指针比较。当前实现会为每个 induction 扫描函数中的候选地址，最坏为  $O(L(N + U))$ ，实际改写数量受固定上限约束。

### Policy 量化

| 规则                                               | 当前值                       |
|--------------------------------------------------|---------------------------|
| 每循环最多建立的 recurrence                              | 8                         |
| 允许通过旋转暴露退出优化的地址数                                 | 2                         |
| 每个 outer loop 可跨越 inner pointer PHI 的 recurrence | 1                         |
| 同一块中允许追踪的 pointer PHI                            | 2                         |
| 共享 recurrence 的常量 byte offset                    | 4 字节对齐且位于 $[-2048, 2047]$ |

## 2.22 Loop Load Elimination

**算法描述** 当前实现匹配一个或两个基本块组成的旋转循环，要求存在 preheader、唯一 latch、唯一 store、没有 call，并且 header 中的 load 在每次迭代都执行。load 与 store 必须基于同一个步长为 +1 或 -1 的 pointer PHI，且 store offset 与 load offset 之差恰好等于 pointer step，因此第  $k$  轮 store 的地址正是第  $k + 1$  轮 load 的地址。改写在 preheader 保留一次初始 load，在 header 建立 value PHI，并把回边 incoming value 设为本轮 store 的值，循环内原 load 随后删除。每个候选循环进行线性扫描，复杂度为  $O(N + U)$ 。

图 11 中，`p.next` 在下一轮成为新的 `p`，所以本轮写入 `p.next` 的 `y` 就是下一轮原 load 应得到的值。改写后只有首轮从内存读取，后续各轮直接从 PHI 取得上一轮的 `y`。

#### 变换前

```
loop:
  %p = phi ptr [%base, %pre],
             [%p.next, %loop]
  %x = load i32, ptr %p
  %y = add i32 %x, 1
  %p.next = gep i32, ptr %p, 1
  store i32 %y, ptr %p.next
```

#### 变换后

```
pre:
  %x0 = load i32, ptr %base
loop:
  %p = phi ptr [%base, %pre],
             [%p.next, %loop]
  %x = phi i32 [%x0, %pre],
             [%y, %loop]
  %y = add i32 %x, 1
  %p.next = gep i32, ptr %p, 1
  store i32 %y, ptr %p.next
```

图 11: Loop Load Elimination 用 loop-carried PHI 转发上一轮 store 的值。

## 2.23 常量算术 Strength Reduction

**算法描述** 对 signed i32 常量除法/取余，使用 Granlund–Montgomery magic multiplier 构造 `smulh + ashr + sign fixup`；余数再由  $r = x - qd$  得到 [4]。2 的幂留给后端生成压力更低的专用序列。单次 magic 选择使用定宽大整数运算，总体为  $O(N)$ 。



## 2.24 带秩递推表格化 (RRT)

**算法描述** RRT 根据递归函数的控制流和参数变化识别有限状态递推，不匹配函数名或“背包”等源码形式。设函数参数被划分为 rank  $r$ 、会随递归调用改变的整数状态  $\mathbf{s} = (s_1, \dots, s_k)$  和在所有 self call 中保持不变的 context  $\mathbf{c}$ ，表格状态记为  $\mathbf{x} = (r, \mathbf{s})$ ，并取  $\rho(\mathbf{x}) = r$ 。当前实现要求每个 self call 的 rank 实参均为  $r - d$ ，其中  $d$  是正整数常量，因此每条递归边都满足

$$\rho(\mathbf{x}') < \rho(\mathbf{x}),$$

从而 rank 的递增顺序就是递归依赖的拓扑顺序。根调用  $\mathbf{q} = (q_0, \dots, q_k)$  定义候选稠密域

$$D(\mathbf{q}) = \prod_{j=0}^k \{0, \dots, q_j\}.$$

这个矩形域不必在编译期证明闭合，生成代码会在每个被替换的 self call 前检查子状态是否仍位于  $D(\mathbf{q})$ ，失败时回退到原递归函数。该变换属于受限的递归表格化，与经典动态规划和递归 tabulation 的基本思想一致 [1, 2]。

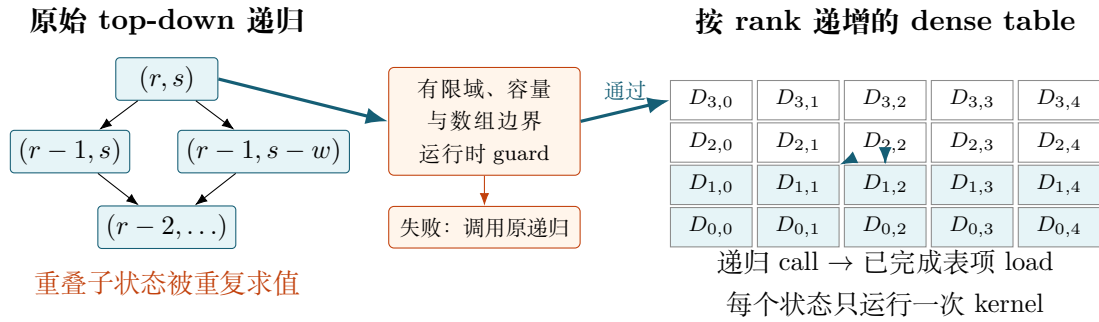


图 12: RRT 将重叠的 top-down 递归变为带 guard 的 bottom-up 状态表。

分析器只接受以下可审计子类：

- 函数返回 i32，CFG 无循环，包含至少两个参数个数匹配的 direct self call，并且模块内存在调用该函数的外部 call site。
- 函数体只包含纯整数算术、比较、扩展、PHI、分支、返回和稳定的 i32 global load，不允许 store、未知 call 或其他具有可观察效果的指令。
- 至少一个 i32 参数在每条递归边上减去正整数常量，第一个满足条件的参数作为唯一 rank。
- 数组读取只接受 `global[rank-1]` 形态，并要求访问块被证明 rank 为正的分支支配，使稠密枚举额外状态时不会产生负下标。

lowering 保留原函数不变，并生成同签名 helper、容量为 131,072 个 i32 的编译器 scratch global 和  $k + 1$  层状态循环。模块中原先从其他函数调用递归函数的 call site 被重定向到 helper，原函数只供 fallback 使用。helper 的核心结构可概括为

Listing 1: RRT lowering 的控制结构

```
if root guard fails:
    return original(root, context)
for r = 0 .. root.r:
```

```

for s1 = 0 .. root.s1:
    ...
    table[index(r, s1, ...)] =
        cloned_kernel(r, s1, ..., context)
return table[index(root)]

```

原递归 CFG 只克隆一次, domain 参数替换为循环 PHI, context 参数保持不变。每个 return 改为把结果写入当前表项并进入下一状态, 每个 self call 改为子状态边界检查和已完成表项的 load。行主序扁平索引为

$$I(x_0, \dots, x_k) = (\dots((x_0 e_1 + x_1) e_2 + x_2) \dots) e_k + x_k, \quad e_j = root_j + 1.$$

helper 先检查每个根状态满足  $0 \leq q_j < 131072$ , 检查 rank 不超过任何 `global[rank-1]` 数组的长度, 并用逐维除法检查  $\prod_j (q_j + 1) \leq 131072$ , 从而在乘法前排除容量溢出。克隆 kernel 中的每个 self call 还检查  $0 \leq x'_j \leq q_j$ 。任一根 guard 或子状态 guard 失败都会调用完整保留的原递归函数求根结果。若  $P$  为参数数、 $C$  为 self call site 数, 识别成本为  $O(N + E + U + PC)$ , 生成代码量为  $O(N + kC)$ 。成功路径执行  $|D|$  次非递归 kernel, scratch 空间固定为 131,072 个 i32。

**加速比** 表 5 给出单 Pass、同提交 QEMU A/B。三个 `knapsack_naive-*` 的动态指令分别减少 15,674.25、35,143.83 和 22,719.89 倍, 几何平均约为  $23,217.42\times$ 。最小值来自 `knapsack_naive-1`, 最大值来自 `knapsack_naive-2`。

| 测试点                           | 关闭 RRT        | 开启 RRT  | 指令数加速比            |
|-------------------------------|---------------|---------|-------------------|
| <code>knapsack_naive-1</code> | 2,415,919,058 | 154,133 | $15,674.25\times$ |
| <code>knapsack_naive-2</code> | 4,630,515,610 | 131,759 | $35,143.83\times$ |
| <code>knapsack_naive-3</code> | 2,415,919,058 | 106,335 | $22,719.89\times$ |

表 5: RRT 单 Pass 的 QEMU 动态指令 A/B (同提交、同 IR 管线)

### 3 MIR 层优化与寄存器分配

IRToMachineLowering 将已经过中端优化的 SSA IR 降为带类型虚拟寄存器的 Machine IR。MIR 保留显式基本块和 CFG, 操作数分为虚拟寄存器、立即数、符号、栈槽和基本块引用, 指令则使用 ADD、LOAD、CONDBR 等尚未完全展开为 RISC-V 汇编的 Machine opcode。刚完成 lowering 时 MIR 仍是 SSA 形式, 因此 Copy Propagation 可以直接替换虚拟寄存器, 随后 PHI Elimination 在 CFG 边上生成复制并退出 SSA。之后的机器级优化以目标立即数字段、寄存器压力和 ABI 约束为依据, 最后由寄存器分配、frame lowering 和 Emitter 生成 RV64GC 汇编。

表 6 给出后端的实际执行次序。Block Placement 在寄存器分配前后各运行一次, 第一次为常见路径创造 fallthrough, 第二次整理 Branch Folding 删除中转块后的布局。

以下复杂度使用  $M$ 、 $B$ 、 $E$ 、 $V$  和  $U$  分别表示 MIR 指令数、基本块数、CFG 边数、虚拟寄存器数和寄存器操作数使用数,  $P$  表示 PHI incoming edge 数。

| 阶段     | Pass 次序与目的                                                                                        |
|--------|---------------------------------------------------------------------------------------------------|
| 退出 SSA | Copy Propagation → PHI Elimination                                                                |
| 局部机器优化 | Memory Address Folding → Machine CSE → Global Merge                                               |
| 循环与常量  | Machine LICM → Loop Condition Duplication → Machine Constant CSE → Global Address Materialization |
| 布局与分配  | Machine Block Placement → Iterated Register Allocation                                            |
| 分配后清理  | Machine Branch Folding → Machine Block Placement                                                  |

表 6: MIR 优化与寄存器分配的实际管线

### 3.1 Machine Copy Propagation

**算法描述** 该 Pass 在 freshly lowered、仍满足 SSA 的 MIR 上扫描可合并的一元复制。相同类型的 MOVE 可以直接传播，i1 到整数寄存器类的 ZEXT 不改变已经规范化的布尔表示，i32 到 i64 的 SEXT 也可复用已经由 RV64 word 运算或 load 完成符号扩展的源值。匹配成功后，Pass 将目标虚拟寄存器的所有使用改为源寄存器并删除原指令。由 live-range splitting 显式标记为不可合并的复制保持不变。当前实现为每条被删除复制扫描全部使用，最坏复杂度为  $O(MU)$ 。

例如，`%b = move %a; %r = add %b, 1` 被改写为 `%r = add %a, 1`。这一早期传播缩短 live range 并减少后续 PHI 复制和干涉图节点。

### 3.2 PHI Elimination

**算法描述** 对每个含 PHI 的基本块，Pass 按前驱边收集  $destination \leftarrow incoming$  复制，并在对应边的末端实现一组 parallel copy。若前驱还有其他后继，复制不能无条件放在前驱中，Pass 会拆出专用 edge block，再将原分支的目标重定向到该块。只有一个后继的前驱可以直接在 terminator 前插入复制，自环回边也使用这一形式，因为新的 PHI 值只在下一轮迭代生效。parallel copy 优先发射不会覆盖其他待用源值的复制，遇到  $a \leftarrow b, b \leftarrow a$  一类环时生成临时虚拟寄存器，将其顺序化为  $t \leftarrow b, b \leftarrow a, a \leftarrow t$ 。收集工作为  $O(P + B + E)$ ，朴素的安全复制选择在单条边上最坏为  $O(P^2)$ 。

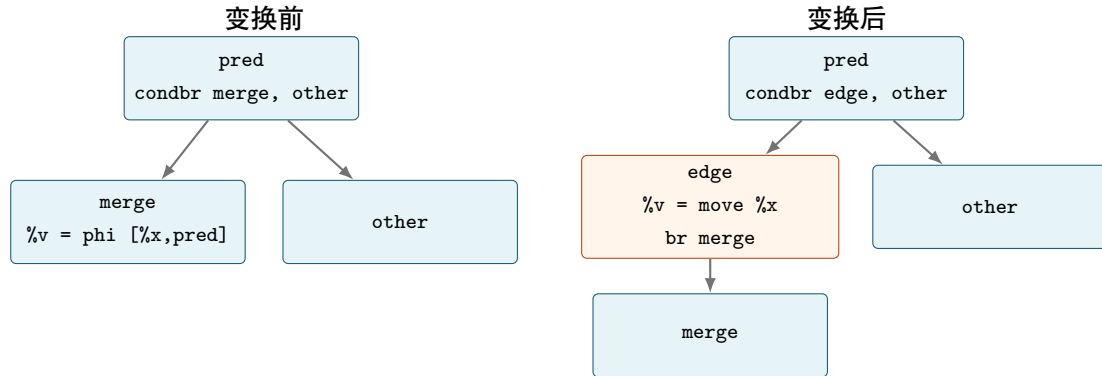


图 13: PHI Elimination 在多后继前驱上拆边并放置复制

### 3.3 Memory Address Folding

**算法描述** Pass 先建立虚拟寄存器使用表，再匹配只有一个使用的 PTR `ADD(base, off)`。当  $off \in [-2048, 2047]$  且唯一使用是尚未带 displacement 的 LOAD 地址或 STORE 地址时，它把基址和立即数直接写入访存指令并删除 ADD。例如 `%p = add.ptr %base, 24; %v = load %p` 变为 `%v = load 24(%base)`。扫描和重写为  $O(M + U)$ 。

### 3.4 Machine CSE

**算法描述** Machine CSE 为每个基本块维护  $(opcode, type, operands)$  到已有虚拟寄存器的表达式表，当前仅处理 ADD/SUB/MUL/SMULH/SHL/ASHR/AND/XOR/ZEXT/SEXT。若基本块只有一个前驱，且该前驱已完成扫描，表会沿这条唯一前驱链继续传递，因此能够跨越无关 call 和基本块边界复用纯算术结果。汇合块会清空可用表，load、store、call、branch 和其他有副作用或隐含状态的指令从不参与 CSE。表达式查询本身近似  $O(M + U)$ ，当前每次命中会扫描函数替换使用，因此最坏为  $O(MU)$ 。

典型变换为 `%a = mul %x, 4; br body` 与 `body: %b = mul %x, 4` 合并为一次乘法，并将 `%b` 的使用改为 `%a`。如果 `body` 有两个前驱则不传播，因为现有实现没有在 MIR 上构造支配树或 PHI translation。

### 3.5 Global Merge

**算法描述** 这里的“Merge”指地址基址复用，并不改变全局对象本身的声明、大小或汇编布局。Pass 按最终发射顺序分别计算可写段和只读段中各全局对象相对本段首对象的精确字节偏移，再匹配 `ADD.ptr @global, %index`。若两个地址属于同一段、使用同一个 index 虚拟寄存器，第二个地址的所有使用都只是 load/store，且加上对象间位移后仍落在 signed 12-bit 范围内，则删除第二次地址计算并把位移折入访存指令。可用地址沿唯一前驱链传播，汇合点重新开始。布局和候选扫描为  $O(M + U)$ ，当前为验证全部使用而扫描函数，最坏为  $O(MU)$ 。

| 变换前                              | 变换后                                   |
|----------------------------------|---------------------------------------|
| <code>%p = add.ptr @a, %i</code> | <code>%p = add.ptr @a, %i</code>      |
| <code>%x = load %p</code>        | <code>%x = load 0(%p)</code>          |
| <code>%q = add.ptr @b, %i</code> | <code>%y = load sizeof(@a)(%p)</code> |
| <code>%y = load %q</code>        | <code>%q 及第二次地址计算被删除</code>           |

### 3.6 Machine LICM

**算法描述** Machine LICM 当前专门处理循环内重复的高成本整数常量物化，而不是任意机器指令外提。它把源 IR 的自然循环、preheader 和精确 trip count 映射到对应 MIR 块，收集 ICMP/SMULH/CONDBR/ADD/SUB/MUL/AND/XOR 中需要多条 RISC-V 指令生成的立即数，并在外层可用的 preheader 中建立一次 `CONST_INT`。乘以二次幂仍留给后续 shift 强度削减，指针 ADD 只在位移超出 signed 12-bit 时外提。外提前重新计算 liveness，用循环内峰值活跃整数数目限制

新增 live range, 含 call 的循环还按可用 callee-saved 寄存器进一步收紧。循环分析与扫描近似  $O(M + E + U)$ , liveness 固定点的最坏成本取决于 CFG 迭代次数。

### 3.7 Loop Condition Duplication

**算法描述** Pass 匹配只含一条 CONDBR 的空 header。入口前驱仍跳到 header 执行首次判定, 所有从 header 可达且以无条件 BR header 结束的回边块则把 terminator 替换为 header 条件的副本, 使下一轮可以直接跳到 body 或 exit。之后的 Block Placement 把常走的 body 放在回边之后, 从而消除“latch 跳到空 header, 再由 header 分支”的额外跳转。候选 header 各自进行一次可达性搜索, 若有  $H$  个候选, 最坏复杂度为  $O(H(B + E))$ 。

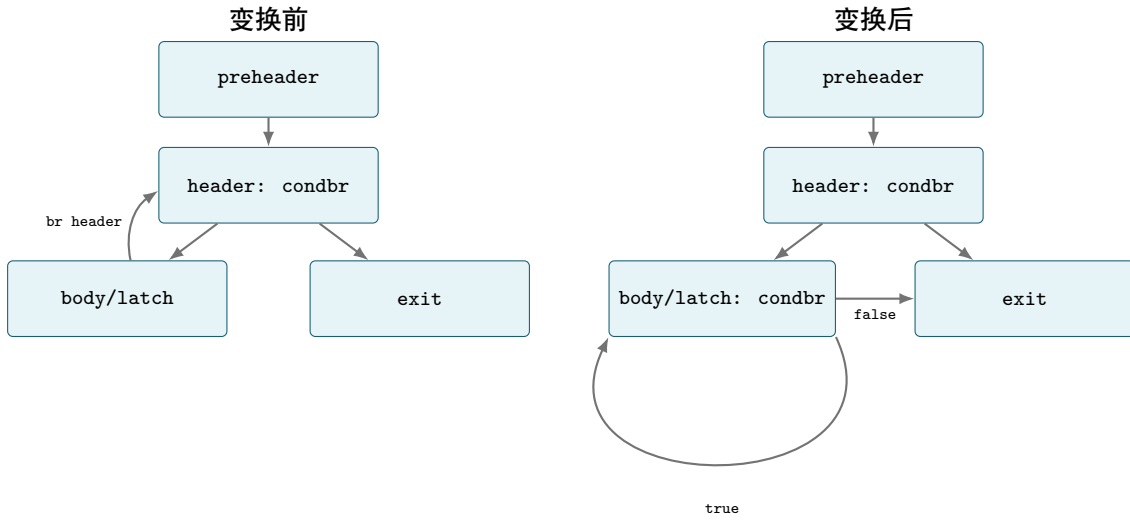


图 14: Loop Condition Duplication 保留首次 header 判定并把后续判定复制到回边

### 3.8 Machine Constant CSE

**算法描述** 该 Pass 将多次隐式立即数展开改为一次显式 CONST\_INT 加寄存器复用。同一基本块内, 对超出 signed 12-bit 的 ADD 立即数至少出现三次时共享一次物化。对于低 12 位非零、通常需要 lui+addi 的高成本常量, 只要父块和唯一前驱的直接后继都使用同一  $(type, value)$ , 也可在父块物化并供相邻块共享。汇合点、非唯一前驱和更远的 CFG 区域不会扩展 live range。扫描为  $O(M + E + U)$ 。

### 3.9 Global Address Materialization

**算法描述** Pass 统计每个符号操作数的静态使用次数, 并把循环块中的一次使用按三次计权, 因为一次 la 外提可以从每个动态迭代中消失。达到阈值且至少在非入口块出现的符号, 会在函数入口生成一个 MOVE @symbol 地址虚拟寄存器, 函数内同名字号操作数全部改用该寄存器。候选按加权使用数排序, 并由 liveness 估计的剩余整数寄存器数截断。若地址跨 call 活跃, 可用数进一步限制为剩余 callee-saved 寄存器数。递归函数中, 只有一次循环引用却跨 self call 的地址不会被外提, 避免每次调用为一个长 live range 支付保存成本。当前循环块搜索对每个块进行可达性检查, 最坏为  $O(B(B + E))$ , 其余统计和替换为  $O(M + U)$ 。

### 3.10 Machine Block Placement

**算法描述** Block Placement 从 entry 开始建立简单 trace，每次按照 terminator 中的目标顺序选择第一个尚未放置的后继，CONDBR 的 true target 先于 false target。当前 trace 结束后，从原始顺序中最早的未放置块开始下一条 trace，直到所有块恰好出现一次。例如原顺序 entry, header, exit, body 在 header 的 true target 为 body 时会变为 entry, header, body, exit。当前实现不读取 profile 或估算块频率，列表查找使最坏复杂度为  $O(B^2 + E)$ 。

Pass 在寄存器分配前运行一次，使循环 body 和偏好的分支目标尽可能成为 fallthrough，又在 Branch Folding 后运行一次，整理删除中转块后的顺序。Emitter 根据最终相邻关系反转条件或省略无条件跳转，具体规则见表 7。

### 3.11 迭代式图着色寄存器分配

**算法描述** ACCELA 采用基于迭代寄存器合并的图着色算法进行寄存器分配。分配器在 Machine IR 上分析虚拟寄存器的活跃关系，并构建寄存器干涉图。随后，分配器通过图简化、复制合并、冻结、溢出选择和颜色分配，将虚拟寄存器映射到 RISC-V 物理寄存器。当物理寄存器不足时，分配器为溢出变量创建栈槽并重写 Machine IR，之后重新执行活跃变量分析和图着色。

#### 前置条件

寄存器分配在 PHI 消除之后执行。此时 Machine IR 中不允许继续存在 PHI 指令，所有 PHI 节点已经被转换为控制流边上的复制指令。寄存器分配器以单个 Machine Function 为单位进行处理。

#### 活跃变量分析

分配器首先计算每个基本块的使用集合 USE 和定义集合 DEF，然后利用逆向数据流分析计算：

$$\text{LiveOut}[B] = \bigcup_{S \in \text{Succ}(B)} \text{LiveIn}[S],$$

$$\text{LiveIn}[B] = \text{USE}[B] \cup (\text{LiveOut}[B] - \text{DEF}[B]).$$

分配器反复逆序遍历基本块，直到所有基本块的活跃集合不再变化。随后，分配器从基本块末尾向前扫描，计算每条指令执行前后的 LiveBefore 和 LiveAfter 集合。

#### 干涉图构建

干涉图中的节点表示虚拟寄存器，边表示两个虚拟寄存器不能被分配到同一个物理寄存器。对于定义寄存器  $d$  的指令  $I$ ，分配器将  $d$  与  $\text{LiveAfter}[I]$  中仍然活跃的同类寄存器连接：

$$v \in \text{LiveAfter}[I] \implies (d, v) \in E.$$

整数寄存器和浮点寄存器使用不同的物理寄存器文件，因此二者之间不建立干涉边。

## 复制指令收集

对于复制指令  $d \leftarrow s$ ，构图阶段暂不添加  $d$  与  $s$  之间的干涉边，并将该复制加入待处理复制集合。后续阶段尝试将二者合并，使其获得相同的物理寄存器，从而消除不必要的复制指令。

## 工作表初始化

分配器根据节点的度数和复制相关性，将虚拟寄存器划分到不同工作表：

- 简化工作表：度数小于  $K$ ，并且不与复制相关；
- 冻结工作表：度数小于  $K$ ，但与复制相关；
- 溢出工作表：度数大于或等于  $K$ ；
- 复制工作表：保存尚未完成合并判断的复制指令。

其中， $K$  表示该虚拟寄存器可选择的物理寄存器数量。

## 图简化

分配器从简化工作表中选择一个度数小于  $K$  的节点，将其暂时移出干涉图并压入选择栈。节点移除后，其相邻节点的有效度数相应减少，部分高阶节点可能因此变为低阶节点并进入简化工作表。

## 复制合并

如果复制指令两端的节点不相同、二者之间不存在干涉边，并且合并后仍满足保守可着色条件，分配器便将两个节点合并。ACCELA 使用 Briggs 保守条件，即合并后邻接节点中度数不小于  $K$  的节点数量必须小于  $K$ 。

## 冻结

如果某个复制暂时无法安全合并，分配器可以冻结相关复制关系。被冻结的复制不再参与合并判断，其相关节点可以作为普通节点进入图简化阶段。冻结操作牺牲复制消除机会，以保证分配过程继续推进。

## 溢出选择

当不存在可以简化、合并或冻结的节点时，分配器从溢出工作表中选择一个潜在溢出节点。其选择代价为：

$$\text{SpillCost}(v) = \frac{\text{WeightedReferences}(v)}{\max(1, \text{Degree}(v))}.$$

循环中的引用按照  $10^{\text{LoopDepth}}$  加权。因此，循环越深、使用越频繁的变量越不容易被选中，而干涉度数较高的变量更容易成为溢出候选。

## 颜色分配

图简化结束后，分配器按照选择栈的逆序恢复节点。对于每个节点，先收集其已经着色的相邻节点所占用的物理寄存器，再从候选寄存器集合中选择一个可用颜色。如果没有可用颜色，该节点被标记为溢出。



## 函数调用约束

如果一个虚拟寄存器在函数调用前后都保持活跃，则该寄存器跨越调用活跃。此类整数变量只能分配到 `s1--s11`，浮点变量只能分配到 `fs0--fs11`，以避免被被调用函数破坏。

## 溢出重写

分配器为溢出节点创建栈槽，在每次使用之前插入地址计算和加载指令，并在每次定义之后插入地址计算和存储指令。重写产生新的虚拟寄存器，因此必须重新执行活跃变量分析、干涉图构建和图着色。ACCELA 最多执行 20 轮溢出重写。

## 结果写回

分配成功后，分配器记录虚拟寄存器到物理寄存器或栈槽的映射，并收集函数实际使用的被调用者保存寄存器。后续栈帧布局阶段为溢出栈槽和保存寄存器分配空间，汇编生成阶段再根据映射输出物理寄存器。

## 4 RISC-V Codegen

### 4.1 指令选择与局部规则

表 7: RISC-V 汇编生成与窥孔规则

| 规则             | 变换                                                                                                                                                                                                                   |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 立即数算术与移位       | $x + c$ 、 $x - c$ 、 $x \text{ xor } c$ 、 $x \text{ and } c$ 和常量移位分别生成 <code>addi/addiw</code> 、 <code>xori</code> 、 <code>andi</code> 、 <code>slli/slliw</code> 或 <code>srai/sraiw</code> 。立即数必须可编码，减法还要求 $-c$ 可编码。  |
| 整数取负           | $0 - x$ 生成 <code>neg/negw</code> ，要求左操作数是精确整数零。                                                                                                                                                                      |
| 二次幂乘法          | $x * 2^k$ 生成 <code>slli/slliw</code> ，64 位模式要求 $k < 64$ ，word 模式要求 $k < 32$ 。                                                                                                                                        |
| 双置位幂乘法         | 若正常量可写成 $2^a \pm 2^b$ ，则 $x * (2^a \pm 2^b)$ 生成至多两次 <code>shift</code> 和一次 <code>add/sub</code> 。                                                                                                                    |
| 二次幂除法          | signed i32 $x/2^k$ 生成符号 <code>bias</code> 和 <code>sraiw</code> ，除数必须是正的 2 次幂，结果保持 toward-zero 舍入。                                                                                                                    |
| 二次幂取余          | signed i32 $x \bmod 2^k$ 生成符号 <code>bias</code> 、 <code>mask</code> 和 <code>subw</code> ，除数必须是正的 2 次幂。                                                                                                               |
| 零比较            | <code>icmp eq/ne x,0</code> 直接使用 <code>zero</code> 和 <code>seqz/snez</code> 。                                                                                                                                        |
| 有符号范围比较        | 当 $c$ 可表示为 signed 12-bit 时， <code>icmp slt/sge x,c</code> 使用 <code>slti</code> ， <code>sge</code> 再以 <code>xori 1</code> 取反。 <code>icmp sle/sgt x,c</code> 改为比较 $c + 1$ ，要求 $c \neq \text{LONG\_MAX}$ 且 $c + 1$ 可编码。 |
| 常量相等比较         | 当 $-c$ 可表示为 signed 12-bit 时， <code>icmp eq/ne x,c</code> 先生成 <code>addi x,-c</code> ，再生成 <code>seqz/snez</code> 。                                                                                                    |
| 比较与分支融合        | 若 <code>icmp</code> 仅被相邻的 <code>condbr</code> 使用，则直接生成 <code>beq/bne/blt/bge</code> ， <code>sgt/sle</code> 通过交换操作数表达。                                                                                                |
| Fallthrough 选择 | Machine block layout 确定后，若一个 target 是下一打印块，则反转条件使其 fallthrough，并省略多余的 <code>j</code> 。                                                                                                                               |
| 地址折叠           | 若 <code>base+off</code> 仅用于一次 <code>load/store</code> 且 $off \in [-2048, 2047]$ ，则直接生成 <code>lw/sw/flw/fsw off(base)</code> ，见 §3.3。                                                                                 |
| 零值存储           | 非浮点 <code>store</code> 的源是精确整数零时，直接以 <code>zero</code> 为源寄存器。                                                                                                                                                        |
| 冗余 MOVE        | <code>MOVE</code> 的目的与源已分配到同一物理寄存器时，不输出指令。                                                                                                                                                                           |
| 小对象清零          | 固定大小不超过 128 B、大小已知且地址至少满足 i32 数组对齐时，内联连续的 <code>sd zero</code> ，尾部使用 <code>sw zero</code> 。                                                                                                                          |
| 大对象清零          | 固定大小超过 128 B 时，调用汇编文件内的 <code>__accela_memzero</code> 。helper 使用 32 B 展开循环，不依赖 <code>libc</code> 或 <code>GNU assembler</code> 内建。                                                                                    |

中端常量除法的 magic multiply 见 §2.23；它生成 SMULH Machine opcode，Emitter 用 RV64 `mul` 后右移 32 位取得 signed i32 高半积。该做法避免引入 SysY 不存在的源级 i64 类型，同时利用目标寄存器的 64 位乘法结果。

### 4.2 清零 helper 的取舍

小对象全部展开能消除 `call`，但会线性增加代码尺寸；大对象全部调用 helper 则使 64 字节一类对象付出参数移动、`call/return` 和循环 `setup`。当前分界为 128 字节：不超过阈值时优先避免动态分支，超过阈值时使用 ACCELA 自己输出的 `__accela_memzero`。

### 4.3 浮点/整型混合与舍入

i32 转 float 使用 `fcvt.s.w`, 遵循当前动态 rounding mode; float 转 i32 明确输出 `fcvt.w.s` ..., `rtz`, 与 SysY/C 向零截断语义一致。浮点比较使用 `feq.s/flt.s/fle.s,one/une` 由 equality 结果取反实现。浮点寄存器间 copy 使用 `fsgnj.s rd,rs,rs`; 整数参数寄存器承载 float 时以 `fmv.x.w/fmv.w.x` 逐位移动, 不做数值转换。

call 参数提交顺序经过专门约束: 先写 stack 参数, 再提交浮点寄存器参数, 最后提交整数参数, 防止浮点常量物化临时使用整数 scratch 时覆盖已经就位的 `a0--a7`。分配器同时拒绝会造成固定参数寄存器 copy cycle 的 affinity。

## 5 性能与正确性分析工具

### 5.1 分析工具与证据边界

我们在对 ACCELA 做性能分析时, 并不只是比较运行时间, 而是尝试把源语言执行、目标程序动态行为、热点静态吞吐量、真实硬件时间和参考编译器结果进行互相验证。项目为此实现并接入了五类工具:

1. AST 解释器直接执行经过 Sema 的 SysY AST, 为编译结果提供绕过 AST2IR、优化 Pass 和后端的输出基线, 也用于把错误定位到前端语义或后续编译阶段。
2. QEMU TCG 插桩执行 ACCELA 生成并链接后的 RV64GC binary, 收集动态指令、load/store、热点翻译块和 L1D miss proxy, 回答程序实际执行了哪些代码以及工作量集中在哪里。
3. llvm-mca 对 QEMU 定位出的热点汇编建立静态调度模型, 基于 SiFive P550、Rocket Core 和受双发射宽度约束的乱序模型拟合 BOOMv3, 比较依赖链、吞吐量与执行资源压力。
4. 侧信道一致性分析联合负载源码、输入规模、生成的 .s、QEMU 事件和真实评测时间反推循环次数、工作集、IPC 与合理耗时区间, 用于优化合法性检验。
5. LLVM -O3 作为独立参考编译器生成同一 RV64GC 目标, 通过输出、动态指令、访存和热点结构的横向比较判断 ACCELA 生成的代码质量。

### 5.2 AST 解释器与差分正确性基线

我们为 ACCELA 开发了一个内置的 AST 解释器, 它在 Sema 完成名字绑定、类型推导和显式转换后直接执行 AST, 覆盖全局与局部变量、整浮点表达式、数组和初始化列表、函数调用、条件与循环控制流以及 SysY I/O。它与编译路径共享 Lexer、Parser 与 Sema, 但不经过 AST2IR、IR/MIR Pass、寄存器分配和汇编生成, 因此解释器输出与目标 binary 输出不一致时, 可以先判断错误是否进入了 IR 之后的阶段。

### 5.3 QEMU TCG 动态分析工具

我们在 QEMU System mode 上实现了三种面向 RV64GC binary 的 TCG plugin。profile 统计计时区间内的动态指令、load 和 store 总数, hotblocks 按“基本块指令数乘执行次数”排列热点翻译块, cache 则以 32 KiB、8-way、64 B cache line 和 LRU 替换模拟 L1D 工作集。三者

分别回答“实际执行了多少指令”、“时间主要花在哪里”和“访存是否可能留在一级缓存”这三个问题，组合后使我们可以区分指令膨胀、重复 load/store、循环次数变化与容量型 cache 问题。

QEMU 分析的是 ACCELA 最终生成并链接后的 ELF，而不是 IR 指令的抽象计数。程序输出必须先与参考输出逐字节一致，性能事件才进入统计。裸机运行时用保留 marker 界定 main 或源码显式 starttime/stoptime 区间，并排除会受宿主机调度影响的 SysY I/O 轮询，QEMU 事件因此对固定 binary 和输入具有确定性。

#### 5.4 llvm-mca 与 BOOM v3 成本估算

QEMU 的热点块地址首先被映射回 ACCELA 生成的 RISC-V 汇编，随后只对实际高频的循环主体做 llvm-mca 分析。P550 模型提供乱序 RISC-V 核心的资源压力和吞吐量参照，Rocket 模型提供顺序核心的依赖执行参照，受双发射宽度约束的通用乱序模型用于观察前端带宽上限。根据估算模型给出的数据分析 load/store 端口、整数乘除单元、分支和 loop-carried dependency 中哪一项限制 block throughput。由于 LLVM 22.1.8 没有 BOOM v3 的现成调度模型，故 ACCELA 使用 QEMU 的动态基本块次数对静态热点加权，再用 50 MHz 实验设备的真实时间、已知的双发射上限以及往届设备经验估算 BOOM v3 的有效 IPC 和 cache/branch penalty。

ACCELA 在开发中的每个性能结论都要求至少两层证据相互支持。AST 解释器、官方输出和 LLVM -O3 结果用于确认语义，QEMU 指令和访存事件说明动态工作量，热点汇编与 llvm-mca 解释吞吐量来源，真实硬件时间最终决定优化是否有效。若真实时间变化无法由动态工作量和热点结构解释，或输入规模、指令数与耗时不按同一趋势变化，该测试点就进入侧信道复核，而不会直接被写成加速比。

#### 5.5 ACCELA 与 LLVM -O3 的 60 点横评

定义

$$R_I = \frac{N_{\text{LLVM -O3}}}{N_{\text{ACCELA}}}.$$

$R_I > 1$  表示 ACCELA 执行的动态指令更少。表 8 同时给出全部 60 点和排除三个 RRT 点后的统计，后者用于观察普通中端与后端质量，防止算法级递推变换支配几何平均。

| 集合        | ACCELA 胜/负 | 几何平均 $R_I$ | 中位数 $R_I$ | 总指令比  |
|-----------|------------|------------|-----------|-------|
| 全部 60 点   | 31/29      | 1.653      | 1.020     | 1.745 |
| 排除 RRT 三点 | 28/29      | 1.028      | 1.000     | 1.129 |

表 8: ACCELA 与 LLVM -O3 的 QEMU 动态指令汇总

全部 60 点累计执行 8,665,743,874 条 ACCELA 指令与 15,118,771,083 条 LLVM 指令；load+store 分别为 1,144,727,804 与 3,913,603,277。后者的 3.419 倍总量差同样被 RRT 强烈放大；排除三个递推点后，动态指令和内存操作总量比分别为 1.129 和 1.133。图 15 的上图保留全部 60 点并使用对数轴，下图放大其余 57 点。普通点围绕 1.0 分布，many\_mat\_cal-\* 达到 1.704–1.717，O3\_sort\* 达到 1.227–1.228；LLVM 最明显领先的是 h-8-\*，其  $R_I = 0.675$ –0.676。

表 9 给出完整的 instruction/load/store 原始计数。

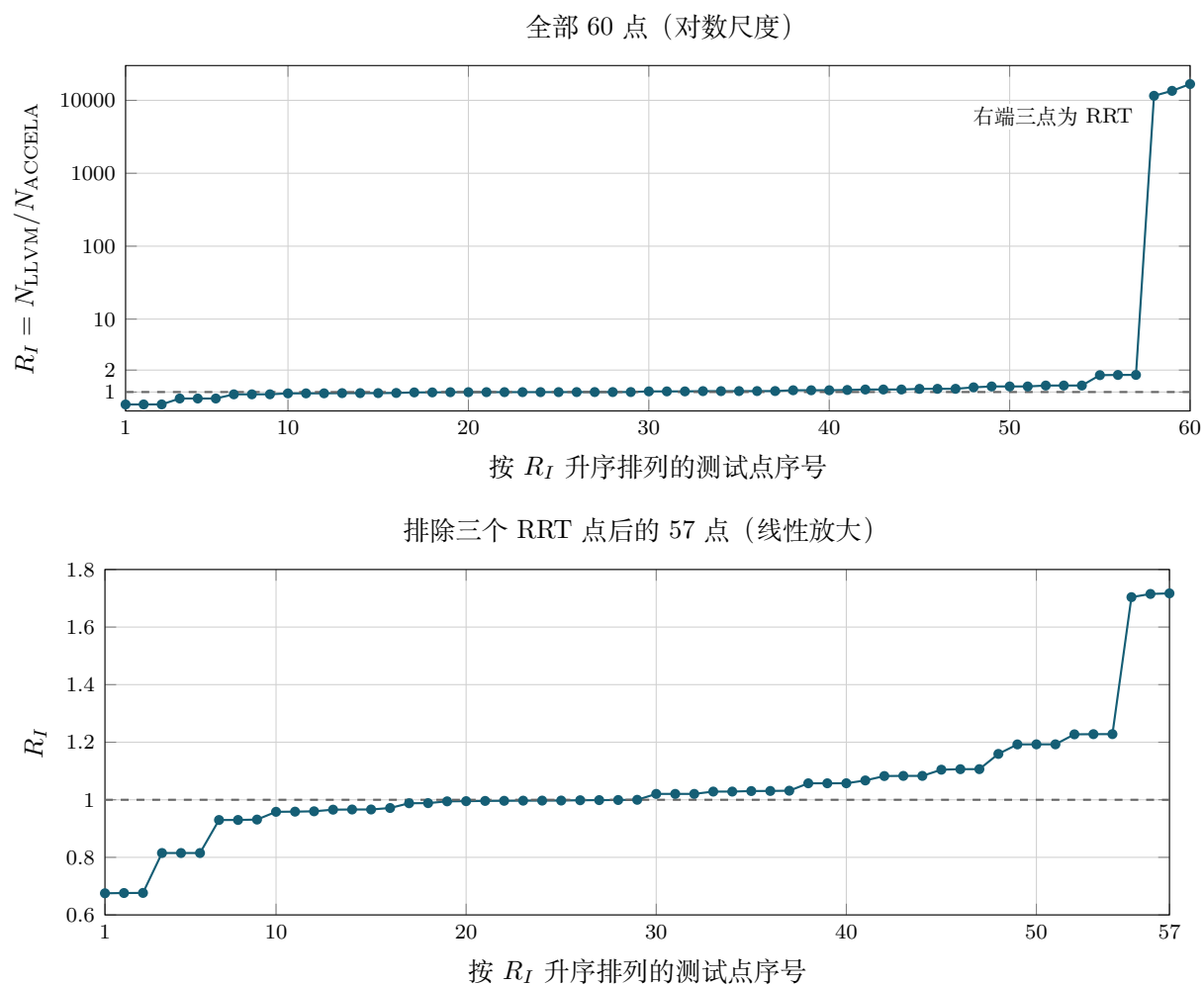


图 15: 60 点 LLVM/ACCELA 动态指令比分布。横轴为按  $R_I$  升序排列后的测试点序号；虚线  $R_I = 1$  表示两者相同。

表 9: ACCELA 与 LLVM -03 的完整 60 点 QEMU 事件

| 测试点      | ACCELA 指令     | LLVM 指令       | $R_I$ | ACCELA load | LLVM load  | ACCELA store | LLVM store |
|----------|---------------|---------------|-------|-------------|------------|--------------|------------|
| 01_mm1   | 81,311,418    | 80,989,912    | 0.996 | 20,018,000  | 20,018,000 | 10,054,000   | 10,054,000 |
| 01_mm2   | 273,640,998   | 272,911,562   | 0.997 | 67,714,500  | 67,714,500 | 33,958,500   | 33,958,500 |
| 01_mm3   | 178,503,488   | 177,955,782   | 0.997 | 44,102,500  | 44,102,500 | 22,127,300   | 22,127,300 |
| 03_sort1 | 72,745,946    | 89,341,241    | 1.228 | 6,768,830   | 4,798,512  | 10,036,964   | 5,870,150  |
| 03_sort2 | 72,806,191    | 89,359,060    | 1.227 | 6,779,600   | 4,805,252  | 10,060,718   | 5,882,437  |
| 03_sort3 | 72,719,699    | 89,296,317    | 1.228 | 6,765,983   | 4,796,215  | 10,030,585   | 5,866,773  |
| conv2d-1 | 1,572,629,218 | 1,703,155,797 | 1.083 | 14,595,366  | 14,866,819 | 1,085,789    | 1,086,056  |
| conv2d-2 | 429,172,558   | 464,789,473   | 1.083 | 3,962,568   | 4,036,564  | 295,961      | 296,104    |
| conv2d-3 | 141,474,902   | 153,142,305   | 1.082 | 1,380,486   | 1,406,419  | 103,709      | 103,796    |
| crc1     | 204,271,992   | 243,617,764   | 1.193 | 250,500     | 250,500    | 0            | 0          |
| crc2     | 204,312,025   | 243,617,764   | 1.192 | 250,500     | 250,500    | 0            | 0          |
| crc3     | 204,278,299   | 243,617,764   | 1.193 | 250,500     | 250,500    | 0            | 0          |
| crypto-1 | 263,855,423   | 269,242,127   | 1.020 | 10,423,000  | 10,423,901 | 4,022,505    | 4,021,910  |
| crypto-2 | 184,698,803   | 188,469,497   | 1.020 | 7,296,100   | 7,296,731  | 2,815,755    | 2,815,340  |
| crypto-3 | 105,542,183   | 107,696,867   | 1.020 | 4,169,200   | 4,169,561  | 1,609,005    | 1,608,770  |
| fft0     | 7,653,408     | 7,894,688     | 1.032 | 622,536     | 822,440    | 622,280      | 822,181    |
| fft1     | 82,085,793    | 84,424,442    | 1.028 | 6,620,814   | 8,731,150  | 6,618,766    | 8,729,099  |
| fft2     | 35,996,172    | 37,023,312    | 1.029 | 2,917,495   | 3,839,556  | 2,916,471    | 3,838,529  |
| h-1-01   | 29,102,058    | 27,901,452    | 0.959 | 0           | 1          | 0            | 0          |
| h-1-02   | 2,375,127     | 2,280,041     | 0.960 | 0           | 1          | 0            | 0          |
| h-1-03   | 168,620,279   | 161,574,168   | 0.958 | 0           | 1          | 0            | 0          |
| h-10-01  | 15,128,205    | 15,055,923    | 0.995 | 3,705,975   | 3,705,975  | 1,883,250    | 1,883,250  |
| h-10-02  | 35,528,480    | 35,401,548    | 0.996 | 8,748,300   | 8,748,300  | 4,428,000    | 4,428,000  |
| h-10-03  | 69,009,855    | 68,810,673    | 0.997 | 17,044,125  | 17,044,125 | 8,606,250    | 8,606,250  |
| h-4-01   | 25,881,529    | 26,665,822    | 1.030 | 0           | 0          | 0            | 0          |
| h-4-02   | 12,329        | 12,707        | 1.031 | 0           | 0          | 0            | 0          |
| h-4-03   | 379,598,631   | 375,035,320   | 0.988 | 0           | 0          | 0            | 0          |
| h-5-01   | 32,047        | 26,120        | 0.815 | 6,350       | 6,341      | 440          | 430        |
| h-5-02   | 32,047        | 26,120        | 0.815 | 6,350       | 6,341      | 440          | 430        |
| h-5-03   | 32,047        | 26,120        | 0.815 | 6,350       | 6,341      | 440          | 430        |

续下页

表 9 (续)

| 测试点                      | ACCELA 指令   | LLVM 指令       | $R_I$     | ACCELA load | LLVM load   | ACCELA store | LLVM store  |
|--------------------------|-------------|---------------|-----------|-------------|-------------|--------------|-------------|
| h-8-01                   | 325,833     | 220,411       | 0.676     | 71,002      | 48,966      | 5,300        | 5,313       |
| h-8-02                   | 325,281     | 219,614       | 0.675     | 71,002      | 48,966      | 5,133        | 5,146       |
| h-8-03                   | 325,816     | 220,263       | 0.676     | 71,002      | 48,966      | 5,389        | 5,402       |
| h-9-01                   | 516,268     | 480,051       | 0.930     | 0           | 6,000       | 0            | 6,000       |
| h-9-02                   | 344,732     | 320,512       | 0.930     | 0           | 4,000       | 0            | 4,000       |
| h-9-03                   | 425,811     | 396,523       | 0.931     | 0           | 5,000       | 0            | 5,000       |
| huffman-01               | 329,938,008 | 318,796,011   | 0.966     | 3,174,000   | 252,000     | 3,636,000    | 582,000     |
| huffman-02               | 330,164,008 | 318,816,011   | 0.966     | 3,170,000   | 252,000     | 3,640,000    | 586,000     |
| huffman-03               | 328,966,008 | 317,872,011   | 0.966     | 3,154,000   | 252,000     | 3,620,000    | 584,000     |
| knapsack_naive-1         | 154,133     | 1,778,647,012 | 11539.690 | 14,783      | 469,762,042 | 3,926        | 402,653,178 |
| knapsack_naive-2         | 131,759     | 1,778,388,964 | 13497.286 | 12,659      | 469,762,042 | 3,328        | 402,653,178 |
| knapsack_naive-3         | 106,335     | 1,778,450,404 | 16724.977 | 10,183      | 469,762,042 | 2,704        | 402,653,178 |
| many_mat_cal-1           | 293,325,043 | 503,675,861   | 1.717     | 89,624,832  | 142,075,728 | 678,976      | 678,976     |
| many_mat_cal-2           | 294,458,094 | 501,746,435   | 1.704     | 89,697,051  | 141,386,877 | 675,684      | 675,684     |
| many_mat_cal-3           | 288,470,253 | 494,757,745   | 1.715     | 88,252,500  | 139,691,100 | 672,400      | 672,400     |
| matmul1                  | 105,498,807 | 111,537,146   | 1.057     | 28,164,038  | 28,164,038  | 160,000      | 160,000     |
| matmul2                  | 354,412,300 | 374,762,925   | 1.057     | 94,893,048  | 94,893,048  | 360,000      | 360,000     |
| matmul3                  | 205,420,790 | 217,188,430   | 1.057     | 54,921,902  | 54,921,902  | 250,000      | 250,000     |
| optimization_scheduling1 | 86          | 85            | 0.988     | 0           | 0           | 0            | 0           |
| optimization_scheduling2 | 1,321       | 1,320         | 0.999     | 0           | 0           | 0            | 0           |
| optimization_scheduling3 | 13,021      | 13,020        | 1.000     | 0           | 0           | 0            | 0           |
| shuffle0                 | 5,518,245   | 5,488,393     | 0.995     | 945,028     | 945,031     | 540,270      | 640,270     |
| shuffle1                 | 29,464,813  | 31,443,141    | 1.067     | 5,400,981   | 5,396,617   | 3,700,300    | 4,700,300   |
| shuffle2                 | 6,349,406   | 6,339,068     | 0.998     | 877,696     | 876,337     | 992,914      | 1,159,580   |
| s11                      | 163,224,144 | 180,597,908   | 1.106     | 46,613,556  | 46,614,150  | 15,762,392   | 23,762,395  |
| s12                      | 320,033,444 | 354,047,408   | 1.106     | 91,579,456  | 91,580,200  | 30,877,992   | 46,502,995  |
| s13                      | 20,062,044  | 22,158,908    | 1.105     | 5,656,756   | 5,657,050   | 1,941,192    | 2,941,195   |
| transpose0               | 38,032,325  | 37,988,130    | 0.999     | 2,414,151   | 2,414,151   | 4,953,182    | 4,928,182   |
| transpose1               | 70,763,128  | 68,739,173    | 0.971     | 4,696,035   | 4,696,035   | 9,538,752    | 9,509,383   |
| transpose2               | 565,921,468 | 656,096,512   | 1.159     | 27,466,551  | 27,466,551  | 56,066,702   | 55,839,902  |



## 6 AI 使用与优化合法性分析

### 6.1 AI 使用说明

本项目使用 Codex 辅助文献检索与阅读、相关编译器源码对照,以及循环优化的局部实现、测试生成和文档整理。优化算法的初始设计、适用条件与正确性论证均由项目成员完成,对于 Codex 生成的实现,项目成员进行了逐项审查、修改与验证。

在性能优化过程中,项目成员结合前述 QEMU 动态指令分析工具与中间表示进行人工分析,定位潜在的 Bad IR Shape,并据此针对性地调整相关中端 Pass。Codex 主要承担检索、实现辅助和重复性工程工作,最终的技术方案、代码取舍与正确性责任均由项目成员负责。

以下是几个我们与 AI 交互中的 Case Study:

**循环展开与融合。** 以矩阵运算和批量点积中常见的循环为例,原程序可能具有如下形式:

```
for (int j = 0; j < m; ++j) {
    int sum = 0;
    for (int k = 0; k < n; ++k)
        sum += a[k] * b[k][j];
    c[j] = sum;
}
```

项目成员首先确定 Loop Unroll-and-Jam 的合法性条件,包括不同 j 迭代之间不存在写后读或写后写依赖、每个展开通道内部保持原有 k 迭代顺序、展开因子不造成过高寄存器压力,以及不能整除时必须生成 remainder loop。在这些约束下,我们给出了一种潜在的可能简化表示:

```
int j = 0;
for (; j + 3 < m; j += 4) {
    int s0 = 0, s1 = 0, s2 = 0, s3 = 0;
    for (int k = 0; k < n; ++k) {
        int x = a[k];
        s0 += x * b[k][j];
        s1 += x * b[k][j + 1];
        s2 += x * b[k][j + 2];
        s3 += x * b[k][j + 3];
    }
    c[j] = s0;
    c[j + 1] = s1;
    c[j + 2] = s2;
    c[j + 3] = s3;
}
for (; j < m; ++j) {
    int sum = 0;
    for (int k = 0; k < n; ++k)
        sum += a[k] * b[k][j];
    c[j] = sum;
}
```

Codex 在这一过程中用于查阅 LLVM 中的循环依赖分析和展开实现,并辅助生成 Loop Unroll-and-Jam、Loop Interchange 及共享仿射依赖分析的初始代码和测试。我们负责检查依赖方向、PHI 节点更新、live-out 值、remainder loop 以及寄存器压力等关键问题,并根据生成 IR 和实测结果修改实现。

**递归表格化。** 对于有限、纯且具有良基秩的递归,项目成员先定义了变换所需的证明条件:递归函数不能具有可观察副作用,状态域必须有限,每条递归边必须严格降低某个秩,数组下标必须在所有新增计算状态上安全,无法证明时必须回退到原递归。

例如,二项式系数可以写成如下纯递归:

```
int binomial(int n, int k) {
    if (k < 0 || k > n)
        return 0;
    if (k == 0 || k == n)
        return 1;
    return binomial(n - 1, k - 1)
        + binomial(n - 1, k);
}
```

其递归依赖关系为:

```
B(n, k) -> B(n - 1, k - 1)
B(n, k) -> B(n - 1, k)
```

两个递归调用都会严格减小参数  $n$ , 因此可以选择  $\rho(n, k) = n$  作为良基秩, 并按照  $n$  递增的顺序计算状态:

```
for (int n = 0; n <= N; ++n) {
    for (int k = 0; k <= K; ++k) {
        if (k > n)
            table[n][k] = 0;
        else if (k == 0 || k == n)
            table[n][k] = 1;
        else
            table[n][k] =
                table[n - 1][k - 1] +
                table[n - 1][k];
    }
}
```

我们完成了一个初步的算法描述,包括输入,输出,算法的伪代码实现,三个不同例子下算法的理论表现与 SSA 形式下的正确性证明,并写了一份超过 15 页的 PDF, Codex 根据上述设计辅助实现递归分析、原 CFG 克隆、多维循环生成、递归调用到 table load 的替换,以及运行时边界检查。项目成员在审查中发现,稠密表格化会计算原递归调用树中不存在的状态,因此仅检查原

始执行路径上的数组合法性并不充分。最终实现增加了对  $i > 0$  控制流支配关系的证明，并在状态域不闭合（例如出现负重量）时调用原始递归函数。

**利用 QEMU 和 IR 定位 Bad IR Shape。** 项目成员使用以下方式对单个 Pass 进行开关 A/B 测试：

```
QEMU_PROFILE=1 scripts/qemu-run.sh testcase.sy

ACCELA_DISABLE_LOOP_UNROLL=1 \
QEMU_PROFILE=1 scripts/qemu-run.sh testcase.sy
```

QEMU 工具记录动态指令、load 和 store 数量。Pass instrumentation 同时记录各阶段的 IR 指令、GEP、load 和 store 数量。随后人工对原始 IR、优化后 IR 和最终汇编进行对照。例如，项目成员在一次扩大 LoopUnroll 适用范围的实验中，虽然功能测试全部通过，但 conv2d-1 的 main 函数由 198 条 IR 指令增长到 234 条，并额外产生了 24 条 store 和 24 条 GEP；QEMU 也观察到明显的动态指令和内存操作回退。其典型 IR 可示意为：

```
%next = add i32 %current, %delta

%addr.0 = getelementptr ..., %index.0
store i32 %next, ptr %addr.0
...
%addr.1 = getelementptr ..., %index.1
%current.1 = load i32, ptr %addr.1
```

项目成员据此没有使用这一实验的版本，而是基于得到的 IR 继续检查展开候选、SROA、Mem2Reg 和后续清理 Pass 之间的交互。

## 6.2 Pass 的合法性分类

表中将各项分为“主流编译器通用优化”和“ACCELA 自研优化”。前一类表示相同的算法思想已广泛用于现代优化编译器，并不表示 ACCELA 调用、绑定、移植或逐条复刻了某个编译器的实现，后一类表示其具体变换形式由 ACCELA 自行设计并给出通用性证明。

表 10: Pass 的算法类别与合法性依据

| Pass 名                                   | 分类                     | 依据                                                                               |
|------------------------------------------|------------------------|----------------------------------------------------------------------------------|
| 支配树/Loop/IndVar<br>PostDom/ModRef 分析     | 主流编译器通用优化              | 支配关系、循环结构、归纳变量、后支配关系以及别名和内存效应是现代优化编译器的基础分析框架, ACCELA 的分析范围见 §2.1。                |
| SROA                                     | 主流编译器通用优化              | 将不逃逸的聚合对象拆为标量是通用的标量化手段, ACCELA 额外要求数组仅由常量索引访问, 见 §2.2。                           |
| Mem2Reg                                  | 主流编译器通用优化              | 基于 dominance frontier 插入 PHI 并沿支配树重命名是标准的 SSA promotion 算法, 见 §2.3。              |
| EarlyCSE                                 | 主流编译器通用优化              | 基于值编号和内存版本消除局部公共子表达式与重复 load 是通用优化, ACCELA 使用对象级 ModRef, 见 §2.4。                 |
| SCCP                                     | 主流编译器通用优化              | 稀疏条件常量传播是经典的 SSA 数据流优化, 见 §2.5。                                                  |
| InstSimplify/InstCombine                 | 主流编译器通用优化              | 代数恒等式化简与指令规范化是通用的局部优化, ACCELA 的定宽语义检查见 §2.6。                                     |
| SimplifyCFG                              | 主流编译器通用优化              | CFG 折叠、分支线性化、diamond 化简和 tail merge 均是通用控制流优化, 见 §2.7。                           |
| ADCE/GlobalDCE<br>Dead Store Elimination | 主流编译器通用优化<br>主流编译器通用优化 | 基于活跃性和调用关系删除不可观察代码是通用死代码消除方法, 见 §2.8。<br>在值被读取前由后续写完全覆盖时删除旧写是标准的死存储消除条件, 见 §2.9。 |
| GVN                                      | 主流编译器通用优化              | 沿支配关系进行全局值编号并使用 PHI translation 处理汇合点是通用冗余消除方法, 见 §2.10。                         |
| GlobalOpt                                | 主流编译器通用优化              | 基于全程序使用信息简化全局对象并提升参数是常见的过程间优化, 见 §2.11。                                          |
| IPSCCP                                   | 主流编译器通用优化              | 在调用图上传播参数和返回值格并求固定点是通用的过程间常量传播算法, 见 §2.12。                                       |
| Inliner                                  | 主流编译器通用优化              | 克隆被调函数 CFG、替换形式参数并汇合返回值是标准的直接调用内联方法, ACCELA 的成本模型只影响收益选择, 见 §2.13。               |
| TailRecursionElimination                 | 主流编译器通用优化              | 将直接尾递归改写为参数 PHI 和循环回边是经典的尾递归消除方法, 见 §2.14。                                       |
| Loop Interchange                         | 主流编译器通用优化              | 依据方向向量和地址步长决定循环交换的合法性与收益是通用循环优化方法, 见 §2.16。                                      |

| Pass 名                     | 分类          | 依据                                                                               |
|----------------------------|-------------|----------------------------------------------------------------------------------|
| Loop Rotate                | 主流编译器通用优化   | 将入口判定重排为 latch 判定以形成规范循环是通用的循环规范化变换, 见 §2.17。                                    |
| LICM                       | 主流编译器通用优化   | 依据支配、可推测执行和内存效应条件外提循环不变量是经典循环优化, 见 §2.18。                                        |
| Loop Unroll-and-Jam        | 主流编译器通用优化   | 复制外层迭代并融合各 lane 的内层循环体是通用循环变换, 其合法性依赖跨 lane 依赖检查和 remainder 处理, 见 §2.19。         |
| Loop Unroll                | 主流编译器通用优化   | 对精确且较小的常量迭代次数复制循环体是通用展开方法, 见 §2.19。                                              |
| IndVarSimplify/LFTR        | 主流编译器通用优化   | 规范化归纳变量并将退出条件归约到更适合目标代码的递推量是通用循环优化, 见 §2.20。                                     |
| Loop Strength Reduction    | 主流编译器通用优化   | 将循环中的仿射地址表达式改写为逐次递推是经典强度削减, 见 §2.21。                                             |
| Loop Load Elimination      | 主流编译器通用优化   | 通过循环携带值转发消除可由上一轮 store 确定的 load 是通用的 store-to-load forwarding, 见 §2.22。          |
| 常量算术 Strength Reduction    | 主流编译器通用优化   | 用乘法、移位和修正序列替换常量除法与取余是通用后端优化, 算法采用 Granlund-Montgomery 方法, 见 §2.23。               |
| RRT                        | ACCELA 自研优化 | 该变换以语义充分条件识别 direct finite ranked recursion, 其归纳证明、运行时 guard 与 fallback 见 §2.24。 |
| Machine Copy Propagation   | 主流编译器通用优化   | 在保持寄存器约束的前提下传播等价寄存器并删除无效 copy 是通用机器级优化, 见 §3.1。                                  |
| PHI Elimination            | 主流编译器通用优化   | 在 CFG 边上生成 parallel copy 并正确拆解复制环是 SSA 退出阶段的标准方法, 见 §3.2。                        |
| Memory Address Folding     | 主流编译器通用优化   | 将地址计算吸收到目标指令可表达的寻址形式中是通用目标相关优化, 见 §3.3。                                          |
| Machine CSE                | 主流编译器通用优化   | 沿支配关系消除无副作用机器表达式是通用的机器级公共子表达式消除, 见 §3.4。                                         |
| Global Merge               | 主流编译器通用优化   | 合并满足布局和可见性条件的小型全局对象以复用地址基址是通用后端优化, 见 §3.5。                                       |
| Machine LICM               | 主流编译器通用优化   | 将循环不变的机器指令或常量化物外提到 preheader 是通用机器级循环优化, 见 §3.6。                                 |
| Loop Condition Duplication | ACCELA 自研优化 | 该变换仅把无副作用的 header branch 复制到原 backedge, 其适用条件和等价性论证见 §3.7。                       |

| Pass 名                  | 分类        | 依据                                                       |
|-------------------------|-----------|----------------------------------------------------------|
| Machine Constant CSE    | 主流编译器通用优化 | 合并重复常量化并在收益允许时选择共享位置是通用的目标相关常量优化，见 §3.8。                 |
| Machine Block Placement | 主流编译器通用优化 | 根据执行频率组织 trace 并增加 fallthrough 是通用的机器基本块布局优化，见 §3.10。    |
| 图着色寄存器分配                | 主流编译器通用优化 | Iterated register coalescing 是经典寄存器分配算法，见 §3.11。         |
| Machine Branch Folding  | 主流编译器通用优化 | 删除仅含跳转的中转块并重定向前驱是通用的机器 CFG 化简，见 §??。                     |
| RISC-V 指令选择规则           | 主流编译器通用优化 | 立即数选择、零比较、分支融合和算术强度削减属于主流后端共有的目标模式匹配，ACCELA 的完整适用边界见表 7。 |

### 6.3 部分测试点优化结果复核

#### 静态审计

源码名称扫描使用：

```
rg -i 'knapsack|conv2d|crypto|h-8|huffman|transpose|many_mat|matmul|shuffle' \
src/main/java/accela/pass src/main/java/accela/backend
```

在 4c54d7b 上无输出，即优化 Pass 和后端没有测试名称分支。RRT 生成的 `__rrt_<function>_values` 只是匹配成功后的唯一符号名，分析过程从未比较函数名。

计时调用也做了独立审计：AST2IR 把 `starttime/stoptime` 降为 `_sysy_starttime/_sysy_stoptime` direct call，GlobalModRefAnalysis 将两者标为 `observable`，ADCE/CSE/LICM 不能当纯函数删除或移动。

#### h-8-\*

**现象** 排行榜将三个点的 ACCELA 用时显示为 0.00，因此对该结果需要解释程序是否被整体删除。

**算法规模与缓存工作集** 以 h-8-01 的复核记录为例，我们测量出计时区间实际执行 327,631 条动态指令。

源码固定  $n = 50$ 。外层  $i, j$  遍历  $\binom{50}{2} = 1,225$  个区间，最内层  $k$  循环总共执行  $\binom{50}{3} = 19,600$  次，这与 QEMU 首要热点块的 19,600 次执行完全一致。计时区间只访问 `seq[0..49]` 和 `table[0..49][0..49]`，有效数据分别为 200 B 和 10,000 B。考虑 64 B cache line 以及二维数组每行 1,400 个整数的步长，50 段表数据占 200 条 cache line，序列占 4 条，总工作集约 13,056 B。32 KiB、8-way、64 B 的 L1D 代理共记录 76,302 次数据访问，其中只有 204 次 miss，miss rate 为 0.27%，且 miss 数恰好等于首次触及的 204 条 cache line，因此该工作集在预热后可驻留于 L1D。

**IPC 与耗时估算** 50 MHz 双发射处理器每周期最多提交两条指令，所以仅由发射宽度得到的周期下界和耗时下界为

$$C_{\min} = \left\lceil \frac{327631}{2} \right\rceil = 163816, \quad T_{\min} = \frac{163816}{50 \times 10^6} \approx 3.28 \text{ ms}.$$

从反汇编提取的主热点是每轮 13 条指令的最内层循环，包含 3 条 load、1 条整数乘法和 2 条条件分支。使用 `generic-ooo` 资源模型并以 `-dispatch=2` 限制为双发射后，`llvm-mca` 对该热点给出 1.97 IPC 和 6.5 cycle 的 block throughput，对四路展开的取模后处理循环给出 1.99 IPC。该模型假定 L1 命中和正确分支预测，不能充当 BOOM v3 的精确周期模型，但可以说明热点没有明显的执行端口瓶颈，吞吐量主要受双发射宽度约束。对任意假设的平均 IPC，端到端计算时间近似为

$$T \approx \frac{327631}{50 \times 10^6 \times \text{IPC}}.$$



| 平均 IPC        | 估计周期    | 估计耗时    |
|---------------|---------|---------|
| 2.00 (发射宽度上限) | 163,816 | 3.28 ms |
| 1.97 (热点静态估计) | 166,310 | 3.33 ms |
| 1.80          | 182,017 | 3.64 ms |
| 1.60          | 204,769 | 4.10 ms |
| 1.31 (显示阈值附近) | 250,100 | 5.00 ms |

同时, ACCELA 的指令数仍比 LLVM -O3 多约 48%, 故我们认为这个测试点上的 0.0s 是正常的。

#### crypto-\*

**现象** 三个点的排行榜用时分别为 2.87、2.01 和 1.15 秒, 为排行榜榜首。

**工作量缩放** 三个点使用完全相同的源码, 输入仅改变随机种子和 `rounds`, 其轮数分别为 100、70 和 40。每轮先用有状态伪随机序列写入 32,000 个整数, 再对缓冲区执行 64 轮分组散列并把结果累积到最终输出, 因此计算量应当与轮数近似线性。QEMU 记录的动态指令数分别为 263,855,423、184,698,803 和 105,542,183, 归一化后每轮分别为 2,638,554.23、2,638,554.33 和 2,638,554.58 条。三点每轮还都执行约 104,230 次 load 和 40,225 次 store, 差异仅来自固定的循环进入和退出开销。

| 测试点      | 轮数  | 每轮耗时     | 每轮动态指令       | 推算 IPC |
|----------|-----|----------|--------------|--------|
| crypto-1 | 100 | 28.70 ms | 2,638,554.23 | 1.839  |
| crypto-2 | 70  | 28.71 ms | 2,638,554.33 | 1.838  |
| crypto-3 | 40  | 28.75 ms | 2,638,554.58 | 1.836  |

**吞吐量与结果依赖** 按 50 MHz 时钟计算, 平均 IPC 可由  $IPC = I / (50 \times 10^6 T)$  反推, 三点均约为 1.84, 低于双发射上限且彼此一致。总耗时、动态指令和访存次数同时按 100:70:40 缩放, 说明每轮生成与散列计算都被实际执行。三个随机种子产生不同的最终五整数输出, 当前 ACCELA 与参考输出逐字节一致。LLVM -O3 在相同输入上执行的动态指令统一多约 2.04%, 两者 load/store 数几乎相同, 差异属于普通指令选择和循环代码质量, 而不是 ACCELA 绕过了数据相关计算。

## 7 结论

## 参考文献

- [1] Richard Bellman. *Dynamic Programming*. Princeton University Press, Princeton, New Jersey, 1957.
- [2] Richard S. Bird. Tabulation techniques for recursive programs. *ACM Computing Surveys*, 12(4):403–417, 1980.

- [3] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 13(4):451–490, 1991.
- [4] Torbjörn Granlund and Peter L. Montgomery. Division by invariant integers using multiplication. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation*, pages 61–72, 1994.
- [5] Mark N. Wegman and F. Kenneth Zadeck. Constant propagation with conditional branches. *ACM Transactions on Programming Languages and Systems*, 13(2):181–210, 1991.